DWD injection and subtraction workflow

Detailed code reviews of simulation, SNR and subtraction

Abstract

Hallowed is the labour that hearkens to the hymn of double white dwarfs. Though their ancient fires have faded, their quiet witness endures: two pale embers bearing their song across the darkness. Blessed are the hands that seek their hidden harmony with patience and humility. For in their celestial dance we glimpse the deep thoughts of the Creator, borne upon the waves of the unseen.

Contents

1 DWD simulation pipeline **1**
1.1 Lines 1–49: import packages and configure the simulation . . . 1
1.2 Lines 50–66: create the orbit and continuous noise . . . 2
1.3 Lines 67–112: prepare the catalogue and full-duration signal measurements . . . 2
1.4 Lines 113–134: combine the measurements, apply TDI, and save . . . 3

2 Detailed code review of eric_snr.py **5**
2.1 Imports, numerical settings and temporary output (lines 1–48) . . . 5
2.2 Step 1: read the catalogue and prepare the frequency grid (lines 50–69) . . . 6
2.3 GPU SNR, waveform batches and foreground covariance (lines 69–135) . . . 7
2.4 Step 2: instrumental covariance and absolute SNRs (lines 137–255) . . . 10
2.5 Step 3: update the unresolved foreground (lines 256–283) . . . 13
2.6 Step 4: final relative SNRs and the 11-column table (lines 284–321) . . . 14
2.7 Record the rounds, publish the result and clean up (lines 321–360) . . . 16

3 Subtraction pipeline: imports and configuration **18**

4 Shared input, observation and result preparation **20**
4.1 Step 3: read the inputs and prepare shared state (lines 58–73) . . . 20

5 Global rounds and shared residual covariance **21**
5.1 Step 4: start a round and estimate covariance (lines 74–95) . . . 21

6 Block preparation and template-bank F-statistics **22**
6.1 Step 5: prepare each selected block and evaluate its templates (lines 96–144) . . . 22
6.2 Evaluate the templates recorded in eric_snr . . . 23

7 Joint block inference and data-based acceptance **23**
7.1 Step 6: run RJMCMC and assess the fitted sources (lines 145–204) . . . 23
7.2 Step 7: associate catalogue labels after inference (lines 205–249) . . . 25
7.3 Step 8: allocate source IDs and record the fits (lines 250–292) . . . 26

8 Combined time-domain subtraction and finalization **27**
8.1 Step 9: subtract the combined time-domain signal (lines 293–328) . . . 27
8.2 Step 10: record the round and decide whether to continue (lines 329–346) . . . 28
8.3 Step 11: save the final results and finish (lines 347–365) . . . 29

9 Mathematical model and normalization **29**
9.1 Raw FFT and the correlated XYZ covariance . . . 30
9.2 Measured covariance smoothing and peak downweighting . . . 30
9.3 Four linear amplitudes and the F projection . . . 31
9.4 Joint amplitude marginalization and the zero-source model . . . 31

10 RJMCMC and waveform helper contracts **32**
10.1 Signed sampled coordinates and count prior . . . 32
10.2 Proposals, temperatures and the selected point estimate . . . 32
10.3 DWD parameter-table GPU interface . . . 32
10.4 Native-expression GPU TDI interface . . . 33

11 Array contracts and HDF5 interpretation **33**
11.1 Array shapes and time-domain units . . . 33
11.2 Root groups and per-round records . . . 33
11.3 A direct result reader . . . 34

12 Verification and measured scope **34**
12.1 Current indexed-range controller . . . 34
12.2 Residual-only screen validation before range selection . . . 35
12.3 Forced full-pipeline noise-block benchmark . . . 35
12.4 Scope of the native comparison . . . 36

13 Review findings and scientific limits **36**
13.1 Search completeness and threshold calibration . . . 36
13.2 Endpoint and covariance health . . . 36
13.3 Shared rounds remain conditional fits . . . 36
13.4 FD proposal assessment and native TD subtraction . . . 36
13.5 Reproducibility, labels and failure recovery . . . 37
13.6 Stopping and residual-background interpretation . . . 37

14 Performance and practical use **37**

15 Scope, source identity and coverage **37**

A Complete primary-source listing **38**

B Complete primary-file coverage map **44**

C Dependency source listings **44**
C.1 Observation producer: eric_simulation_DWD.py . . . 44
C.2 Measured covariance and template F-statistics: script/dwd_blind_search.py . . . 46
C.3 DWD synthesis and native-expression GPU TDI: eric_generate_waveform.py . . . 47
C.4 Variable-count inference: eric_rjmcmc.py . . . 51

D Review reproduction **55**

1 DWD simulation pipeline

The subtraction workflow configured in Section 3 operates on a precomputed DWD observation. This section describes how `eric_simulation_DWD.py` produces that input through population injection and detector simulation.

1.1 Lines 1–49: import packages and configure the simulation

```
# Step 1. Import packages.
import os
for thread_variable in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS', '
    NUMEXPR_NUM_THREADS'):
    os.environ[thread_variable] = '1'
import importlib.metadata
import json
from pathlib import Path
import shutil
import tempfile
import time
import h5py
import lisaorbits
import lisagwresponse
import lisainstrument
import numpy as np
import pytdi
from script.oneblock_observation import pack_observation
from eric_generate_waveform import DWD_simulation_gpu, tdi_gpu
```

Import all the necessary packages.

```
# Step 2. Configure parameter settings.
catalogue_file = Path('./data/LDC2_sangria_training_v2.h5')
observation_file = Path('./data/dwd_observation_full_catalogue.h5')
T_obs = 360. * 86400.
fs = 0.1
L = 2.5e9
N_acc = 3.0e-15
delta_x = 15.0e-12
gpu_batch_size = 10000
minimum_envelope_samples = 512
parameter_fields = dict(
    A='Amplitude', f='Frequency', df='FrequencyDerivative', phi0='InitialPhase',
    iota='Inclination', longitude='EclipticLongitude', latitude='EclipticLatitude',
    psi_bme='Polarization',
)
```

The two paths specify the source catalogue and combined observation file. `T_obs`, `fs`, `L`, `N_acc` and `delta_x` set the observation duration, sampling frequency, arm length, acceleration-noise level and OMS-noise level.

`gpu_batch_size=10000` limits the number of ordinary DWDs processed together on the GPU. `minimum_envelope_samples=512` sets the minimum waveform-envelope resolution; wider chirps use smaller batches. The dictionary maps the eight internal parameter names to their catalogue fields.

```
workspace = Path(tempfile.mkdtemp(prefix='.dwd_simulation_', dir=observation_file.parent))
orbit_file = workspace / 'orbit.h5'
N = int(round(T_obs * fs))
N_day = int(round(86400 * fs))
detector_config = dict(
    T_obs=T_obs, gpu_batch_size=gpu_batch_size,
    minimum_envelope_samples=minimum_envelope_samples, include_padding=True,
```

```
    dt=1 / fs, orbits=str(orbit_file), orbit_dataset='tcb/ltt',
    aafilter=('kaiser', 240., .275 * fs, .725 * fs),
    clock_offsets=0., clock_freqoffsets=0., clock_freqlindrifts=0., clock_freqquaddrifts=0.,
)
```

The temporary workspace holds intermediate files until publication. `N` is the observation sample count, and `N_day` supplies one day of boundary padding. The single `detector_config` dictionary supplies both GPU injection and the native instrument.

1.2 Lines 50–66: create the orbit and continuous noise

```
# Step 3. Simulate instrumental noise with the LISA packages.
lisaorbits.KeplerianOrbits(L=L).write(
    str(orbit_file), t0=-20e6, dt=86400., size=825, mode='w',
)
print('Simulating one continuous ACC+OMS noise realization.', flush=True)
noise_instrument = lisainstrument.Instrument(
    **{key: value for key, value in detector_config.items()
       if key not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', 'include_padding')},
    size=N + 2 * N_day,
    t0=-N_day / fs,
    testmass_asds=N_acc, oms_asds=(delta_x, 0., 0., 0., 0., 0.),
)
noise_instrument.disable_all_noises(excluding={'test-mass', 'oms'})
noise_instrument.disable_dopplers()
noise_data = pytdi.Data.from_instrument(
    noise_instrument.export_numpy_full(),
)
```

The orbit table starts at `t0=-20e6`, uses one-day sampling and contains 825 samples, providing coverage for interpolation and delayed measurements. `noise_instrument` produces one continuous ACC+OMS measurement realization with $N + 2N_{\mathrm{day}}$ samples beginning at $-N_{\mathrm{day}}/f_s$, retaining one padding day at each end. `pytdi.Data.from_instrument` exposes the time-domain noise measurements for later combination.

1.3 Lines 67–112: prepare the catalogue and full-duration signal measurements

```
# Step 4. Simulate waveforms of all DWDs on the GPU.
with h5py.File(catalogue_file) as source:
    dgb = source['sky/dgb/cat'][:]
    igb = source['sky/igb/cat'][:]

source_count = len(dgb) + len(igb)
catalogue = np.empty(source_count, dtype=[('ID', 'i8')] + [
    (field, 'f8') for field in parameter_fields.values()
])
catalogue['ID'] = np.arange(1, source_count + 1, dtype=np.int64)
for field in parameter_fields.values():
    catalogue[field] = np.concatenate((dgb[field][:, 0], igb[field][:, 0]))
del dgb, igb

with h5py.File(workspace / 'population.h5', 'x') as saved:
    saved.create_dataset('catalogues/all', data=catalogue, chunks=True)
```

All catalogue rows are combined into one table with a unique ID and eight physical parameters per DWD. This table is saved for the subsequent injection and subtraction analyses.

```
links_file = workspace / 'population_links.h5'
print(f'Simulating all {source_count:,} DWDs with DWD_simulation_gpu.', flush=True)
```

```
started = time.perf_counter()
links = DWD_simulation_gpu(catalogue, detector_config)
waveform_runtime = time.perf_counter() - started
print(f'GPU injection completed: {source_count:,} DWDs in {waveform_runtime:.1f}s.', flush=True)
del catalogue
response_model = lisagwresponse.GalacticBinary(
    A=1., f=.003, ra=0., dec=0., orbits=str(orbit_file),
    orbit_interp_order=5, shift_sun='never',
)
with h5py.File(links_file, 'x') as saved:
    response_model.init_response_file(saved, detector_config['dt'], len(links), -2 * N_day / fs, 'tcb')
    saved['tcb/y'][:] = links
del links
```

`DWD_simulation_gpu` receives the DWD parameter table and detector settings, then returns the summed six-link time-domain response. It accepts the eight physical parameters directly, so sources need not belong to the LDC catalogue; any ID column is retained only as table metadata. Lines 91–97 store the summed GPU response in the format required by `lisainstrument`.

```
# Convert the summed links into full-duration detector measurements.
signal_instrument = lisainstrument.Instrument(
    **{key: value for key, value in detector_config.items()
       if key not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', 'include_padding')},
    gws=str(links_file), size=N + 2 * N_day,
    t0=-N_day / fs,
)
signal_instrument.disable_all_noises()
signal_instrument.disable_dopplers()
signal_data = pytdi.Data.from_instrument(signal_instrument.export_numpy_full())
signal_measurements = signal_data.measurements
noise_measurements = noise_data.measurements
del noise_data, noise_instrument
```

The summed links are converted into detector measurements over the entire padded observation in one call. At the end of this subsection, `signal_measurements` and `noise_measurements` are ready on the same time grid.

1.4 Lines 113–134: combine the measurements, apply TDI, and save

```
# Step 5. Combine the full observation, apply TDI, and save.
total_measurements = {
    name: signal_measurements[name] + noise_measurements[name]
    for name in signal_measurements
}
tdi_results = {}
for name, measurements in (('total', total_measurements),
                           ('signal', signal_measurements), ('noise', noise_measurements)):
    tdi_results[name] = tdi_gpu(
        measurements, signal_data.args, keep=slice(N_day, N_day + N),
    ) / signal_instrument.central_freq
detector_config['orbits'] = str(observation_file.with_suffix('.orbits.h5'))
with h5py.File(workspace / 'population_tdi.h5', 'x') as saved:
    saved.attrs.update(duration=T_obs, rate=fs)
    saved.attrs['detector_config'] = json.dumps(detector_config)
    for name, value in tdi_results.items():
        saved.create_dataset(name + '_xyz_td', data=value, chunks=(3, min(N_day, N)))

pack_observation(workspace, observation_file, overwrite=True, time_domain_only=True)
shutil.copy2(workspace / 'orbit.h5', observation_file.with_suffix('.orbits.h5'))
shutil.rmtree(workspace)
print(f'Saved {source_count:,} DWDs: {observation_file.resolve()}', flush=True)
```

The matching signal and noise measurements are added before TDI. `tdi_gpu` applies the installed PyTDI `X2/Y2/Z2` expressions with GPU interpolation, reusing the moving-arm delays for the total, signal and noise streams. It retains the native fifth-order delay interpolation and 31st-order measurement interpolation; the implementation and comparison with standard PyTDI are given in Section 10.4 and Section 12. The file stores only the final time-domain TDI arrays for total, signal and noise, together with the DWD table, orbit and detector settings. The observation also stores `duration=T_obs`, `rate=fs`, and the complete `detector_config` as JSON. The subtraction pipeline reads these saved values directly instead of redefining detector settings. The native orbit is also exported as a companion `.orbits.h5` file for the subtraction pipeline, preserving the orbit used to simulate the observation.

2 Detailed code review of eric_snr.py

This section reproduces all 360 lines of `eric_snr.py` in source order. Its four numbered stages prepare the SNR catalogue used by the subtraction pipeline. Line numbers here belong to `eric_snr.py`; the subsequent subtraction walkthrough resumes its own numbering.

2.1 Imports, numerical settings and temporary output (lines 1–48)

```
1  import os
2  for variable in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS', '
       NUMEXPR_NUM_THREADS'):
3      os.environ[variable] = '1'
4  import logging
5  import time
6  from functools import partial
7  from pathlib import Path
8  from tempfile import TemporaryDirectory
9
10 import h5py
11 import jax
12 import jax.numpy as jnp
13 import numpy as np
14 from jax.scipy.linalg import lu_factor, lu_solve
15 from jax.scipy.signal import fftconvolve
16 from tqdm.auto import tqdm
17
18 from script.eric_generate_waveform import PARAMETER_FIELDS, generate_waveform_td
19 from script.spectrum import DetectorFunction
20
21 jax.config.update('jax_enable_x64', True)
22
23 logging.getLogger('lisainstrument.instru.instru_fplan').addFilter(
24     lambda record: not record.getMessage().startswith(
25         'Using default set of locking beatnote frequencies'))
26
27
```

Lines 1–3 set the listed numerical-library thread counts to one before NumPy and the other scientific packages are imported. This is an existing setting in the SNR producer. It controls host-library threads; waveform generation, FFTs and SNR contractions are dispatched through the GPU path.

Lines 4–19 import file handling, timing, progress display, HDF5, NumPy and JAX. `lu_factor` and `lu_solve` factor and solve the three-channel covariance systems. `fftconvolve` smooths the foreground spectrum. The imported `generate_waveform_td` is the generator in `script/eric_generate_waveform.py`. It yields individual DWD XYZ batches; the `DWD_simulation_gpu` function used by the observation pipeline returns summed six-link responses. These are separate interfaces.

Line 21 enables JAX's 64-bit numerical mode. Lines 23–25 suppress one repeated instrument log message about default locking beatnote frequencies. The filter changes which messages are displayed; it does not alter the instrument configuration or the SNR calculation.

```
28 LDC_file = Path('./data/LDC2_sangria_training_v2.h5')
29 orbit_file = Path('./data/lisa_snr_orbits.h5')
30 output_file = Path('./data/ldc_dwd_data.h5')
31 catalogue_groups = ('dgb', 'igb')
32 T_obs = 360 * 86400
33 fs = 0.1
34 N_acc = 3.0e-15
35 delta_x = 15.0e-12
36 SNR_threshold = 7.0
37 covariance_regularization = 1.0e-12  # Add this times trace(S_instrument)/3 to all channels.
```

```
38 catalogue_chunk_size = 65536
39
40 started = time.perf_counter()
41 output_file.parent.mkdir(parents=True, exist_ok=True)
42 temporary_directory = TemporaryDirectory(prefix='ldc_snr_', dir=output_file.parent)
43 work_directory = Path(temporary_directory.name)
44 final_output_file = output_file
45 output_file = work_directory/final_output_file.name
46 background_file = work_directory/'background.h5'
47 history = []
48
```

Lines 28–38 define this producer's input paths and scientific settings. `LDC_file` supplies the original DWD parameters, `orbit_file` supplies the orbit to the waveform and covariance helpers, and `output_file` names the completed SNR result. This script does not read the simulated observation from Section 1.

The observation duration is $T = 360 \times 86400 = 31{,}104{,}000$ seconds and the sampling rate is $f_s = 0.1\,\mathrm{Hz}$. The two noise amplitudes are $3.0 \times 10^{-15}\,\mathrm{m\,s^{-2}}/\sqrt{\mathrm{Hz}}$ for acceleration and $15.0 \times 10^{-12}\,\mathrm{m}/\sqrt{\mathrm{Hz}}$ for optical metrology. `SNR_threshold=7.0` controls membership of the unresolved foreground. The regularization coefficient is applied to the instrumental covariance in lines 151–152. `catalogue_chunk_size=65536` limits the number of catalogue rows handled by each outer file-processing chunk; the imported waveform generator divides that chunk into its own GPU batches.

`catalogue_groups` names the two storage locations in the original LDC file. Both are processed by the same waveform, covariance and SNR operations. Their rows are later written into one numerical `catalogue` table.

Lines 40–48 begin the runtime measurement and prepare a temporary directory inside the output's parent. The producer creates the parent if needed. The completed output path is saved as `final_output_file`; the working `output_file` and `background_file` are then placed under the temporary directory. `history` records one diagnostic row per foreground round. Publication and cleanup occur at the end.

2.2 Step 1: read the catalogue and prepare the frequency grid (lines 50–69)

```
49 # ----------------------------------------------------------------------
50 # 1. Read every DWD catalogue and prepare the native physical frequency grid.
51 # ----------------------------------------------------------------------
52 print('1/4  Read the complete LDC DWD catalogue', flush=True)
53 # Positive FFT bins, excluding DC and Nyquist.
54 first_bin = 1
55 last_bin = int(T_obs*fs)//2-1
56 frequency = np.arange(first_bin, last_bin+1)/T_obs
57 catalogue_sizes = {}
58 with h5py.File(LDC_file, 'r') as ldc:
59     for group in catalogue_groups:
60         catalogue = ldc[f'sky/{group}/cat']
61         catalogue_sizes[group] = len(catalogue)
62 catalogue_offsets = {'dgb': 0, 'igb': catalogue_sizes['dgb']}
63 source_count = sum(catalogue_sizes.values())
64 print(f'     {source_count:,} binaries; one {T_obs/86400:g}-day observation; '
65       f'{len(frequency):,} native bins; '
66       f'GPU TD waveform batches; GPU FFT/SNR and foreground', flush=True)
67
68
```

Lines 54–56 construct the positive Fourier frequencies

$$N = \mathrm{int}(T f_s) = 3{,}110{,}400, \qquad f_k = \frac{k}{T}, \qquad k = 1, \ldots, \frac{N}{2} - 1.$$

The resulting array contains $K = 1{,}555{,}199$ bins with spacing $\Delta f = 1/T \simeq 3.2150 \times 10^{-8}\,\mathrm{Hz}$. DC and the Nyquist bin are excluded. This grid is determined by the full observation duration; it is not the subtraction pipeline's indexed frequency-block grid.

Lines 57–63 read the lengths of the two original catalogue datasets. The first dataset starts at global index zero; the second starts after all rows in the first. These offsets let the waveform generator identify sources consistently while reading the original file. `source_count` is their sum, currently 29,000,000. The final numerical table uses these same global row indices plus one as its saved IDs.

Lines 64–66 report the source count, duration and frequency count. They do not start a second observation or divide the time series into days. Each generated source waveform spans the full T-second observation.

2.3 GPU SNR, waveform batches and foreground covariance (lines 69–135)

```
69 @jax.jit
70 def SNR_gpu(waveform_fd, covariance_factor, T_obs, fs):
71     vectors = waveform_fd.transpose(2, 1, 0)
72     weighted = lu_solve(covariance_factor, vectors)
73     power = jnp.sum((vectors.conj()*weighted).real, axis=(0, 1))
74     return jnp.sqrt(4/(T_obs*fs**2)*power)
75
76
```

Lines 69–74 define the GPU SNR contraction. For a batch containing B sources, `waveform_fd` has shape $(B, 3, K)$. The three entries are the second-generation XYZ channels. Transposing it gives `vectors` with shape $(K, 3, B)$, so all sources can be solved against the same 3×3 covariance matrix at each frequency.

`lu_solve` calculates $\mathbf{S}_k^{-1}\mathbf{H}_b[k]$ using an already computed LU factorization. The sum in line 73 contracts the channel and frequency axes, leaving one scalar per source. The real part removes numerical imaginary residue from the Hermitian quadratic form.

The implemented normalization is

$$\rho_b^2 = \frac{4}{T f_s^2} \sum_{k=1}^{N/2-1} \mathrm{Re}\Big[\mathbf{H}_b[k]^\dagger \mathbf{S}(f_k)^{-1} \mathbf{H}_b[k]\Big]\,. \tag{1}$$

Here $H_{bi}[k] = \sum_{n=0}^{N-1} h_{bi}[n] \exp(-2\pi \mathrm{i} kn/N)$ is the unscaled discrete FFT. Its physical Fourier amplitude is $H_{bi}[k]/f_s$, and the frequency spacing is $1/T$, which gives the factor $4/(Tf_s^2)$ for a one-sided noise covariance. Line 74 returns ρ_b, rather than ρ_b^2. The full covariance includes cross-channel correlations; XYZ SNRs are not combined as three independent noise channels.

```
77 class GPUJobs:
78     def __init__(self, T_obs, fs, orbit_file, first_bin):
79         self.T_obs = T_obs
80         self.fs = fs
81         self.orbit_file = orbit_file
82         self.first_bin = first_bin
83         self.noise_input = None
84         self.noise_covariance = None
85         self.covariance_factor = None
86         self.foreground_lines = None
87
88     def snr(self, dwd_ids, noise_covariance, chunk=None):
89         return self._results(dwd_ids, noise_covariance, None, chunk)
90
91     def foreground(self, dwd_ids, noise_covariance, chunk=None):
92         return self._results(dwd_ids, noise_covariance, SNR_threshold, chunk)
93
```

Lines 77–86 define the state shared by waveform batches. `T_obs`, `fs`, `orbit_file` and `first_bin` preserve the observation setup. The other attributes initially contain `None`: the most recently supplied noise array, its GPU copy, its LU factorization and the accumulated foreground spectral matrices.

Lines 88–92 expose two routes through the same calculation. `snr(...)` passes `threshold=None`, so it evaluates SNRs without adding any source to the foreground accumulator. `foreground(...)` passes `SNR_threshold`, so it evaluates the same SNRs and also adds sources at or below that threshold. Both return an iterator over completed source batches.

```
94      @partial(jax.jit, static_argnums=(0, 3), donate_argnums=(4,))
95      def _process(self, xyz, covariance_factor, threshold, foreground_lines):
96          waveform_fd = jnp.fft.rfft(
97              xyz, n=int(round(self.T_obs*self.fs)), axis=-1)
98          waveform_fd = waveform_fd[
99              :, :, self.first_bin:self.first_bin+len(covariance_factor[0])]
100         snr = SNR_gpu(waveform_fd, covariance_factor, self.T_obs, self.fs)
101         if threshold is not None:
102             unresolved = (snr <= threshold).astype(waveform_fd.real.dtype)
103             foreground_lines += 2/(self.T_obs*self.fs**2)*jnp.einsum(
104                 'bik,bjk,b->kij', waveform_fd, waveform_fd.conj(), unresolved)
105         return snr, foreground_lines
106
```

Lines 94–105 perform the FFT and optional foreground accumulation. `xyz` has shape $(B, 3, N)$. The real FFT acts along time, then lines 98–99 retain the same K positive bins prepared in Step 1. The rectangular full-duration time window is implicit: the code applies no taper before this FFT.

When `threshold` is present, line 102 assigns weight one to sources with $\rho_b \leq 7$ and zero to the others. The threshold equality therefore belongs to the unresolved set. The accumulated one-sided cross-spectral matrix is

$$L_{kij} \leftarrow L_{kij} + \frac{2}{T f_s^2} \sum_{b:\,\rho_b \leq 7} H_{bi}[k] H_{bj}[k]^*. \tag{2}$$

The `einsum` expression sums over sources while retaining frequency and both channel axes, producing shape $(K, 3, 3)$. It preserves each source's XYZ correlations. Separate sources contribute their own spectral matrices; the code does not form inter-source coherent cross terms by summing all waveforms before taking an FFT.

The JIT decorator treats `self` and `threshold` as static arguments. Donating the previous foreground buffer lets JAX reuse its storage when returning the updated accumulator. The `threshold=None` route skips the update and returns the existing accumulator alongside the newly calculated SNRs.

```
107     def _results(self, dwd_ids, noise_covariance, threshold, chunk=None):
108         description = 'Generate full TD XYZ'
109         if chunk is not None:
110             description += f' [{chunk[0]}/{chunk[1]}]'
111         for batch_ids, _, xyz in generate_waveform_td(
112                 self.fs, self.T_obs, dwd_ids, self.orbit_file,
113                 progress_desc=description):
114             if noise_covariance is not self.noise_input:
115                 self.noise_covariance = jax.device_put(
116                     noise_covariance, jax.devices('gpu')[0])
117                 self.covariance_factor = lu_factor(self.noise_covariance)
118                 self.noise_input = noise_covariance
119             if self.foreground_lines is None:
120                 self.foreground_lines = jnp.zeros_like(self.noise_covariance)
121             snr, self.foreground_lines = self._process(
122                 xyz, self.covariance_factor, threshold, self.foreground_lines)
123             jax.block_until_ready((snr, self.foreground_lines))
124             yield batch_ids, np.asarray(snr)
125
```

Lines 107–124 connect the source IDs to the waveform and SNR calculations. The imported generator yields `batch_ids`, sample times and `xyz`; this caller discards the sample-time array because T, f_s and the FFT grid are already fixed. In the inspected generator, the fast route processes batches of up to 64 sources, with shape $(B, 3, N)$. That internal batch size is distinct from the 65,536-row catalogue chunk.

Lines 114–118 reuse the GPU noise array and its LU factors while the caller supplies the same covariance object. A different covariance object, such as the next round's effective covariance, is transferred and factorized again. The code uses object identity for this cache; it does not compare array values.

Lines 119–120 initialize the foreground accumulator once with shape $(K, 3, 3)$. It remains accumulated across catalogue chunks and subsequent foreground-transfer rounds. Line 123 waits for both returned GPU arrays to finish before advancing progress. Line 124 copies only the batch SNR values to a NumPy array for catalogue bookkeeping; the covariance and foreground arrays remain on the GPU.

This interface passes source IDs to the imported generator. The inspected generator reads their parameters from the project's `data/LDC2_sangria_training_v2.h5`; the current default `LDC_file` names that same catalogue. The generator does not receive `LDC_file` as an argument, so its catalogue path must remain aligned with the file used by this producer's row-reading code.

The generator is passed the specified orbit file for the time-dependent source response. The SNR helper does not replace that moving response with a single-frequency template.

```
126     def confusion_covariance(self):
127         with jax.default_device(self.foreground_lines.device):
128             offsets = jnp.arange(-128, 129)
129             kernel = 0.5*(1+jnp.cos(jnp.pi*offsets/128))
130             kernel /= kernel.sum()
131             covariance = fftconvolve(
132                 self.foreground_lines, jnp.broadcast_to(kernel[:, None, None],
133                     (len(kernel), 3, 3)), mode='same', axes=0)
134         return 0.5*(covariance+covariance.conj().swapaxes(-1, -2))
135
```

Lines 126–134 convert the accumulated spectral lines into a smoothed unresolved-foreground covariance. The normalized kernel is

$$w_m = \frac{\frac{1}{2}[1 + \cos(\pi m/128)]}{\sum_{q=-128}^{128} \frac{1}{2}[1 + \cos(\pi q/128)]}, \qquad m = -128, \ldots, 128.$$

There are 257 kernel positions, with zero weight at the endpoints. The nominal smoothing half-width is $128/T \simeq 4.1152 \times 10^{-6}$ Hz.

The convolution runs along frequency only and applies the same kernel to each of the nine XYZ covariance entries. `mode='same'` keeps the output shape $(K, 3, 3)$; its finite-array edges use the convolution's zero extension without a separate edge-weight renormalization. Line 134 explicitly forms $\mathbf{C} = (\mathbf{C} + \mathbf{C}^\dagger)/2$, preserving a Hermitian matrix at every frequency.

The result is a spectral covariance model for unresolved DWDs. It is not a noise time series and is not a residual produced by fitting and subtracting sources from the simulated observation.

2.4 Step 2: instrumental covariance and absolute SNRs (lines 137–255)

```
136 # ------------------------------------------------------------------------
137 # 2. Instrument-only XYZ covariance, then absolute SNRs for every source.
138 # ------------------------------------------------------------------------
139 print('2/4  Instrumental covariance and round 1 absolute SNRs', flush=True)
140 t = T_obs/2
141 with tqdm(total=1, desc='Instrumental covariance', unit='calculation') as progress:
142     detector = DetectorFunction(
143         fs=fs, orbit_file=orbit_file, orbit_interp_order=5)
144     instrument_covariance, _ = detector.noise_covariance(
145         frequency, t,
146         acceleration_asd=N_acc,
147         oms_asd=delta_x,
148         generation='2nd',
149         combination_type='XYZ')
150     progress.update(1)
151 scale = np.trace(instrument_covariance, axis1=-2, axis2=-1).real/3
152 instrument_covariance += covariance_regularization*scale[:, None, None]*np.eye(3)
153 active_chunks = []
154
```

Lines 139–150 obtain the instrumental XYZ covariance from `DetectorFunction`. The call requests second-generation TDI, XYZ combinations, the supplied acceleration and optical-metrology noise amplitudes, and fifth-order orbit interpolation. Its first return value has shape $(K, 3, 3)$; the second return value, signal-filter power, is ignored by this caller.

The covariance helper evaluates the transfer functions at $t = T/2$, one midpoint epoch. The source waveforms span the moving orbit over the full observation, while this instrumental noise weighting is a frequency-dependent matrix evaluated at that epoch. The code does not average the instrumental covariance over all observation times.

Lines 151–152 add a diagonal term

$$\mathbf{S}_{\mathrm{inst},k} \leftarrow \mathbf{S}_{\mathrm{inst},k} + 10^{-12}\frac{\mathrm{Re}\,\mathrm{tr}(\mathbf{S}_{\mathrm{inst},k})}{3}\mathbf{I}_3.$$

This small regularization scales with the mean instrumental channel power at each frequency. It is added before computing absolute SNRs and is therefore also present in later effective covariances. `active_chunks` will collect IDs that initially exceed the threshold.

```
155 # Save settings as datasets; no HDF5 attributes.
156 with h5py.File(output_file, 'x') as output:
157     settings = output.create_group('settings')
158     settings.create_dataset('source_catalogue', data=str(LDC_file.resolve()))
159     settings.create_dataset('observation_duration_s', data=T_obs)
160     settings.create_dataset('detector_sampling_rate_hz', data=fs)
161     settings.create_dataset('frequency_spacing_hz', data=1/T_obs)
162     settings.create_dataset('smoothing_half_width_hz', data=128/T_obs)
163     settings.create_dataset('signal_time_window', data='rectangular full observation')
164     settings.create_dataset('signal_window_mean_square', data=1.)
165     settings.create_dataset('tdi_channels', data='X2,Y2,Z2')
166     settings.create_dataset('snr_threshold', data=SNR_threshold)
167     settings.create_dataset('transfer_factor_threshold', data=1)
168     settings.create_dataset('neighboring_bins', data=128)
169     settings.create_dataset('noise_acceleration_asd', data=N_acc)
170     settings.create_dataset('noise_oms_asd', data=delta_x)
171     settings.create_dataset('covariance_regularization', data=covariance_regularization)
172     settings.create_dataset('waveform_device', data='gpu')
173     settings.create_dataset('post_waveform_device', data='gpu')
174     settings.create_dataset('snr_cpu_threads', data=0)
175
176     status = output.create_group('status')
177     status.create_dataset('complete', data=False)
```

```
178     status.create_dataset('source_count', data=source_count)
179     output.create_group('sky')
180
```

Lines 155–179 create the temporary working HDF5 file and record its settings as datasets. Mode `'x'` creates a new working file in the fresh temporary directory. `settings` stores the source path, duration, sampling, Fourier spacing, smoothing width, rectangular window, channels, threshold, noise amplitudes and regularization.

The device entries describe the waveform and subsequent numerical stages. `snr_cpu_threads=0` records that the SNR work is dispatched to the GPU; it does not mean that host file reading and bookkeeping use no CPU. `transfer_factor_threshold=1` records an integer stopping convention: the iteration ends when fewer than one source transfers, which the loop implements explicitly as `transfer_factor==0`.

`status/complete` begins as false and `status/source_count` records the total number of rows. The `sky` group holds the original catalogue-layout working datasets created in the next block. These compatibility datasets coexist with the final single numerical table; the current subtraction reader uses `catalogue`.

```
181 fields = ['Name', *PARAMETER_FIELDS]
182 number_chunks = sum((size+catalogue_chunk_size-1)//catalogue_chunk_size
183                     for size in catalogue_sizes.values())
184 gpu = GPUJobs(T_obs, fs, orbit_file, first_bin)
185 chunk_index = 0
186 with tqdm(total=source_count, desc='Round 1: all absolute SNRs',
187           unit='DWD', unit_scale=True) as progress:
188     for group in catalogue_groups:
189         dataset_path = f'sky/{group}/cat'
190         group_size = catalogue_sizes[group]
191         with h5py.File(LDC_file, 'r') as ldc:
192             catalogue_dtype = ldc[dataset_path].dtype
193         # Preserve source IDs; store every numerical field without decimal rounding.
194         dtype = np.dtype([('Name', catalogue_dtype.fields['Name'][0])]
195                          + [(name, '<f8') for name in PARAMETER_FIELDS]
196                          + [('SNR_absolute', '<f8'), ('SNR_relative', '<f8')])
197         chunk_shape = (min(4096, group_size), 1)
198         with h5py.File(output_file, 'a') as output:
199             catalogue_group = output['sky'].create_group(group)
200             catalogue_group.create_dataset('cat', shape=(group_size, 1), dtype=dtype,
201                                            chunks=chunk_shape, compression='lzf', shuffle=True)
202
203         for first in range(0, group_size, catalogue_chunk_size):
204             chunk_index += 1
205             last = min(first+catalogue_chunk_size, group_size)
206             # Read one chunk.
207             with h5py.File(LDC_file, 'r') as ldc:
208                 records = ldc[dataset_path].fields(fields)[first:last, 0]
209
210             # Calculate with the files closed.
```

Lines 181–185 choose the original `Name` field and eight physical parameters, count the outer processing chunks, and instantiate one `GPUJobs` object. With 26 million and 3 million input rows, the 65,536-row setting produces 397 and 46 chunks, respectively: 443 chunks in total. This is a file-processing count, not a count of observation segments or simultaneous GPU sources.

Lines 188–201 preserve each input dataset's original `Name` dtype and store the eight parameters and two SNRs as float64 fields. Each compatibility dataset has shape $(n_{\text{group}}, 1)$ and a compound dtype; its named fields are not a numerical second dimension. This explains why the separate numerical $(29{,}000{,}000, 11)$ table is created in Step 4.

`chunk_shape` controls HDF5 storage chunks of at most 4,096 rows. LZF compression and byte shuffling are lossless. The outer `catalogue_chunk_size` separately controls the 65,536-row reads. Lines 203–208 load one chunk's requested fields. The loading scope closes before the waveform and SNR calculations begin in the next block.

```
211             rows = np.empty(len(records), dtype=dtype)
212             for name in fields:
213                 rows[name] = records[name]
214             rows['SNR_relative'] = np.nan
215             dwd_ids = catalogue_offsets[group]+np.arange(first, last)
216             for batch_ids, snr in gpu.foreground(
217                     dwd_ids, instrument_covariance, chunk=(chunk_index, number_chunks)):
218                 rows['SNR_absolute'][batch_ids-catalogue_offsets[group]-first] = snr
219                 progress.update(len(batch_ids))
220             keep = rows['SNR_absolute'] > SNR_threshold
221             active_chunks.append(dwd_ids[keep])
222 
223             # Save the calculated chunk.
224             with h5py.File(output_file, 'a') as output:
225                 output[dataset_path][first:last, 0] = rows
226 
227 active = np.concatenate(active_chunks) if active_chunks else np.empty(0, dtype=np.int64)
228 del active_chunks
229 resolved_count = len(active)
230 history.append(dict(round=1, evaluated=source_count, resolved=resolved_count,
231                     unresolved=source_count-resolved_count, transfer_factor=None))
232 print(f'Round 1: {resolved_count:,} resolved; {source_count-resolved_count:,} initially unresolved
        ', flush=True)
233 
```

Lines 211–215 copy the input parameters and original names into the working rows. `SNR_relative` is temporarily `NaN`, because the foreground model has not yet been fixed. `dwd_ids` contains zero-based global indices, including the second dataset's offset.

Lines 216–221 call `gpu.foreground` with the instrumental covariance alone. The returned values are saved as `SNR_absolute`; the generator's global batch indices are converted back to positions in the current row chunk. At the same time, the GPU helper accumulates sources with absolute SNR at or below seven into the initial foreground. The rows with absolute SNR strictly above seven supply the initial `active` IDs.

Lines 223–225 write the calculated rows with the source file closed. Lines 227–232 join the active-ID chunks and record round 1. Its `evaluated` count is the full source count, `resolved` is the initial active count and `unresolved` is the complement. `transfer_factor=None` marks that there is no earlier resolved set to transfer from in this first pass.

In this producer, *resolved* means above the model's SNR threshold. It does not establish that RJMCMC has identified or fitted those sources. All original parameter rows are retained in the output, regardless of their threshold classification.

```
234 instrument_covariance = gpu.noise_covariance
235 confusion_covariance = gpu.confusion_covariance()
236 with h5py.File(background_file, 'x') as background:
237     settings = background.create_group('settings')
238     settings.create_dataset('observation_duration_s', data=T_obs)
239     settings.create_dataset('detector_sampling_rate_hz', data=fs)
240     settings.create_dataset('frequency_spacing_hz', data=1/T_obs)
241     settings.create_dataset('smoothing_half_width_hz', data=128/T_obs)
242     settings.create_dataset('signal_time_window', data='rectangular full observation')
243     settings.create_dataset('signal_window_mean_square', data=1.)
244     settings.create_dataset('covariance_axes', data='frequency,XYZ,XYZ')
245     settings.create_dataset('neighboring_bins', data=128)
246 
247     status = background.create_group('status')
```

```
248     status.create_dataset('complete', data=False)
249     background.create_dataset('frequency_hz', data=frequency)
250     background.create_dataset('instrument_covariance', data=np.asarray(instrument_covariance),
        compression='lzf')
251     background.create_dataset('initial_confusion_covariance',
252                               data=np.asarray(confusion_covariance),
253                               compression='lzf')
254
```

Lines 234–235 retain the instrumental covariance already copied to the GPU and smooth the initial unresolved spectral accumulator. **Lines 236–253** save a diagnostic `background.h5` inside the temporary directory. Its frequency array has shape $(K,)$; the instrumental and initial confusion arrays have shape $(K, 3, 3)$. `np.asarray` copies them to the host for HDF5 storage.

The background settings describe the frequency grid, window and channel axes. Its completion flag begins as false, and its final covariances and round history are added at the end. The script subsequently deletes this temporary background file when it removes the working directory. The published `ldc_dwd_data.h5` is the SNR catalogue and its settings, status and round history.

2.5 Step 3: update the unresolved foreground (lines 256–283)

```
255 # ------------------------------------------------------------------------
256 # 3. Re-evaluate only resolved sources and accumulate each transfer once.
257 # ------------------------------------------------------------------------
258 print('3/4  Iteratively update the unresolved foreground', flush=True)
259 round_number = 2
260 while True:
261     effective_covariance = instrument_covariance+confusion_covariance
262     evaluated = resolved_count
263     transfer_factor = 0
264     remaining = []
265     with tqdm(total=evaluated, desc=f'Round {round_number}: resolved SNRs',
266               unit='DWD', unit_scale=True) as progress:
267         for batch_ids, snr in gpu.foreground(active, effective_covariance, chunk=(1, 1)):
268             transfer = snr <= SNR_threshold
269             transfer_factor += np.count_nonzero(transfer)
270             remaining.append(batch_ids[~transfer])
271             progress.update(len(batch_ids))
272     active = np.concatenate(remaining) if remaining else np.empty(0, dtype=active.dtype)
273     resolved_count = evaluated-transfer_factor
274     history.append(dict(round=round_number, evaluated=evaluated, resolved=resolved_count,
275                         unresolved=source_count-resolved_count, transfer_factor=transfer_factor))
276     print(f'Round {round_number}: transfer_factor={transfer_factor:,}; '
277           f'{resolved_count:,} resolved; {source_count-resolved_count:,} unresolved', flush=True)
278     if transfer_factor == 0:
279         break
280     confusion_covariance = gpu.confusion_covariance()
281     round_number += 1
282
```

Lines 259–281 iterate from round 2 with

$$\mathbf{S}_{\mathrm{eff}}^{(r)}(f) = \mathbf{S}_{\mathrm{inst}}(f) + \mathbf{C}_{\mathrm{unresolved}}^{(r-1)}(f).$$

Only the currently active sources are re-evaluated. They were above threshold under the previous covariance; adding foreground noise can reduce their SNRs. A source with relative SNR at or below seven transfers to the unresolved set. `transfer_factor` is the number of such sources, and `remaining` collects IDs still strictly above seven.

The same `gpu.foreground` call adds the transferring sources' spectral matrices to the persistent accumulator. Already unresolved sources are absent from `active`, so their contribution is not added

again. The active list can only shrink; this loop has no transition that moves an unresolved source back into it.

Lines 273–277 record the counts for that round. Here `evaluated` is the previous resolved count, and the new resolved count is `evaluated-transfer_factor`. The total unresolved count is `source_count-resolved_count`.

If no source transfers, lines 278–279 stop the iteration. The foreground accumulator has not changed, so the `effective_covariance` calculated at the start of that round is already the final weighting. Otherwise, line 280 smooths the updated accumulator and the next round begins. There is no fixed maximum number of rounds in this script.

2.6 Step 4: final relative SNRs and the 11-column table (lines 284–321)

```
283 # ---------------------------------------------------------------------
284 # 4. With the final foreground fixed, calculate every relative SNR again.
285 # ---------------------------------------------------------------------
286 print('4/4  Final relative SNRs for every LDC DWD', flush=True)
287 catalogue_columns = ('ID', *PARAMETER_FIELDS, 'SNR_absolute', 'SNR_relative')
288 with h5py.File(output_file, 'a') as output:
289     output.create_dataset('catalogue', shape=(source_count, 11), dtype='f8',
290                           chunks=(min(4096, source_count), 11), compression='lzf', shuffle=True)
291     output['settings'].create_dataset('catalogue_columns',
292         data=np.asarray(catalogue_columns, dtype=h5py.string_dtype('utf-8')))
293 # No transfers occurred in the last round, so its covariance is already final.
294 chunk_index = 0
```

Lines 287–292 create the public numerical `catalogue` dataset. Its shape is (`source_count`, 11), currently (29,000,000, 11), with float64 values. Storage chunks contain at most 4,096 complete rows; LZF and shuffling remain lossless. `settings/catalogue_columns` saves the column order as UTF-8 strings.

Column	Saved name	Meaning
0	`ID`	One-based global catalogue row ID.
1	`Amplitude`	Dimensionless gravitational-wave amplitude.
2	`Frequency`	Reference frequency in Hz.
3	`FrequencyDerivative`	Frequency derivative in Hz/s.
4	`InitialPhase`	Initial phase in radians.
5	`Inclination`	Orbital inclination in radians.
6	`EclipticLongitude`	Ecliptic longitude in radians.
7	`EclipticLatitude`	Ecliptic latitude in radians.
8	`Polarization`	Polarization angle in radians.
9	`SNR_absolute`	SNR weighted by instrumental covariance.
10	`SNR_relative`	SNR weighted by final instrumental plus foreground covariance.

Each ID is represented exactly in float64 at this catalogue size. The raw numerical table occupies $29{,}000{,}000 \times 11 \times 8 = 2{,}552{,}000{,}000$ bytes before compression. The next pass fills all its rows; this dataset is not restricted to above-threshold DWDs.

```
295 with tqdm(total=source_count, desc='Final round: all relative SNRs',
296           unit='DWD', unit_scale=True) as progress:
297     for group in catalogue_groups:
298         dataset_path = f'sky/{group}/cat'
299         group_size = catalogue_sizes[group]
300         for first in range(0, group_size, catalogue_chunk_size):
301             chunk_index += 1
302             last = min(first+catalogue_chunk_size, group_size)
303             # Read the previously saved absolute SNRs.
304             with h5py.File(output_file, 'r') as output:
```

```
305                 rows = output[dataset_path][first:last, 0]
306
307             # Calculate with the files closed.
308             dwd_ids = catalogue_offsets[group]+np.arange(first, last)
309             for batch_ids, snr in gpu.snr(
310                     dwd_ids, effective_covariance, chunk=(chunk_index, number_chunks)):
311                 rows['SNR_relative'][batch_ids-catalogue_offsets[group]-first] = snr
312                 progress.update(len(batch_ids))
313
314             merged_rows = np.column_stack((dwd_ids + 1,
315                 *[rows[name] for name in catalogue_columns[1:]]))
316             # Save the calculated chunk.
317             with h5py.File(output_file, 'a') as output:
318                 output[dataset_path][first:last, 0] = rows
319                 output['catalogue'][catalogue_offsets[group] + first:catalogue_offsets[group] +
    last] = merged_rows
320
```

Lines 295–319 re-evaluate every source with the final `effective_covariance`. This full pass is required because initially unresolved sources and sources transferred in earlier rounds have not all been evaluated against that same final covariance.

Lines 303–305 load the previously saved parameters, original name and absolute SNR. `gpu.snr`, rather than `gpu.foreground`, then calculates the final `SNR_relative` without adding to the foreground. The absolute SNR remains the Step 2 result.

Lines 314–315 assemble each numerical row as its global index plus one, the eight parameters, and the two SNRs. This conversion changes the table layout, not the saved parameter or SNR values. Lines 317–319 write both the compatibility rows and the corresponding slice of the single merged table. The original `Name` values remain in the compatibility datasets, while `ID` in the merged table follows global row order.

The subtraction pipeline reads this table directly and retains its rows with column 10 strictly above `SNR_r_threshold`; see Section 4. It does not need to handle the original storage-group names or rebuild IDs. Both SNRs refer to the same full-duration source response, with different covariance weightings as defined in Equation 1.

2.7 Record the rounds, publish the result and clean up (lines 321–360)

```
# Store one value per round in each dataset.
round_numbers = np.array([row['round'] for row in history])
evaluated_counts = np.array([row['evaluated'] for row in history])
resolved_counts = np.array([row['resolved'] for row in history])
unresolved_counts = np.array([row['unresolved'] for row in history])
# Round 1 has no transfers yet; NaN means not applicable.
transfer_counts = np.array([
    np.nan if row['transfer_factor'] is None else row['transfer_factor'] for row in history])
runtime_seconds = time.perf_counter()-started

with h5py.File(output_file, 'a') as output:
    rounds = output.create_group('rounds')
    rounds.create_dataset('round', data=round_numbers)
    rounds.create_dataset('evaluated', data=evaluated_counts)
    rounds.create_dataset('resolved', data=resolved_counts)
    rounds.create_dataset('unresolved', data=unresolved_counts)
    rounds.create_dataset('transfer_factor', data=transfer_counts)

    status = output['status']
    status.create_dataset('resolved_count', data=resolved_count)
    status.create_dataset('runtime_seconds', data=runtime_seconds)
    status['complete'][()] = True

```

Lines 321–329 convert the in-memory round history into arrays. There is one entry per round for its number, evaluated count, resolved count, unresolved count and transfer count. The first transfer entry is saved as `NaN`, matching its *not applicable* meaning in Step 2. The runtime is measured from the producer's initial `started` value.

Lines 331–342 save these arrays under `rounds`, add the final resolved count and elapsed seconds under `status`, and mark the working catalogue complete. At this point all 29 million rows in `catalogue` have their absolute and final relative SNRs. The public output has no per-source waveform or strain dataset; the large individual XYZ batches were used to calculate these scalars.

```
with h5py.File(background_file, 'a') as background:
    background.create_dataset('final_confusion_covariance', data=np.asarray(confusion_covariance),
     compression='lzf')
    background.create_dataset('final_effective_covariance', data=np.asarray(effective_covariance),
     compression='lzf')

    rounds = background.create_group('rounds')
    rounds.create_dataset('round', data=round_numbers)
    rounds.create_dataset('evaluated', data=evaluated_counts)
    rounds.create_dataset('resolved', data=resolved_counts)
    rounds.create_dataset('unresolved', data=unresolved_counts)
    rounds.create_dataset('transfer_factor', data=transfer_counts)
    background['status/complete'][()] = True
# Publish only the completed catalogue, then remove the temporary files.
output_file = output_file.replace(final_output_file)
temporary_directory.cleanup()
print(f'Saved: {output_file.resolve()}', flush=True)
print(f'{resolved_count:,}/{source_count:,} above SNR_r > {SNR_threshold:g}; '
      f'{time.perf_counter()-started:.1f} s', flush=True)
```

Lines 344–355 complete the temporary background diagnostics with the final confusion covariance, effective covariance and round counts. **Lines 356–360** then move the completed working catalogue to `final_output_file`. `Path.replace` replaces an existing result at that destination. Because the temporary directory was created under the output parent, the working and final files use the same filesystem.

After publication, `temporary_directory.cleanup()` removes the temporary directory and `background.h5`. The final messages display the saved catalogue path, resolved count, threshold and elapsed time. The durable result contains `catalogue`, column names and settings, completion status, round history and compatibility catalogue datasets. The numerical table is the interface used by the current subtraction code.

This review reproduces the inspected local and server source, with SHA-256 `7441c8ba7b7e0dc4680e5e4a14b6bb435be07684c68816f300911d46f1b0510e`. The table export and direct reader were checked in the recorded regression tests in Section 12. Adding this review changes the documentation; it does not rerun the 29-million-source SNR calculation.

3 Subtraction pipeline: imports and configuration

The primary reviewed file is `eric_subtraction_all.py`. The pipeline supports the full frequency grid and a restricted subtraction interval. The three input files are the observation's companion orbit, the completed `eric_snr.py` result and the simulated observation. Section 1 describes the producer of that observation. Section 2 reviews the producer of the SNR catalogue.

The main subtraction pipeline is documented as Steps 1–11, with every source line retained. Step numbers in Section 1 refer to the separate observation simulation.

```
'''Step 1: import all the necessary packages.'''
from pathlib import Path
from tempfile import mkdtemp
import hashlib
import json
import shutil
import h5py
import numpy as np
import lisagwresponse
import lisainstrument
import pytdi
from eric_generate_waveform import DWD_simulation_gpu, tdi_gpu
from script.eric_Fstats import (
    build_response, physical_amplitude_coordinates,
    physical_amplitudes, reconstruct_waveform, noise_inner_product,
)
from script.dwd_blind_search import (
    residual_covariance, template_f_statistic_search, amplitude_f_statistic,
)
from script.dwd_bank_association import assign_bank_ids
from eric_rjmcmc import run_eryn_rjmcmc
from script.oneblock_observation import read_checkpoint, write_checkpoint
from script.tool import _save_tree, single_cpu

```

Step 1. Import all the packages required by the subtraction workflow.

```
'''Step 2: define the parameters.'''
orbit_file = Path('./data/dwd_observation_full_catalogue.orbits.h5')
snr_file = Path('./data/ldc_dwd_data.h5')
observation_file = Path('./data/dwd_observation_full_catalogue.h5')
output_file = Path('./data/dwd_subtraction_all.h5')
frequency_range = (0.0001, 0.05)
f_derivative_pn = [-25, -10]
freq_block_width = 1e-05
subtraction_frequency_range = None
frequency_derivative_bounds = 10. ** f_derivative_pn[1] * np.array([-1., 1.])
F_threshold = 16.0
SNR_r_threshold = 7.0
sample_number = (500, 1500)
maximum_sources = 64
```

Step 2. Define the input paths, search limits, detection thresholds and sampling effort.

The defaults below are read from this source snapshot. Sampling effort and a source-capacity setting specify computational work; neither establishes convergence.

Setting	Default	Meaning
`T_obs`, `fs`	From observation file	Saved duration and sampling frequency.
`freq_block_width`	10^{-5} Hz	Width of the fixed carrier-frequency blocks.
`frequency_range`	$[10^{-4}, 0.05]$ Hz	Limits of the indexed frequency grid.

Setting	Default	Meaning
`f_derivative_pn`	`[-25,-10]`	Declared common $\log_{10}\lvert\dot{f}\rvert$ interval for positive and negative derivatives, using values in Hz/s.
`subtraction_frequency_range`	`None`	Requested interval; `None` selects all grid blocks.
`F_threshold`	16	Threshold on $\mathcal{F}$, rather than $2\mathcal{F}$.
`SNR_r_threshold`	7	Saved relative SNR for candidate selection and fitted-source SNR must be strictly larger.
`sample_number`	`(500,1500)`	Burn-in and production cycles for each sampler call.
`maximum_sources`	64	Minimum per-fit sampler capacity.

```
39 with h5py.File(observation_file, 'r') as saved:
40     T_obs = saved['observation'].attrs['duration']
41     fs = saved['observation'].attrs['rate']
42     total_xyz_td = saved['observation/total_xyz_td'][:]
43     detector_config = json.loads(saved['observation'].attrs['detector_config'])
44 detector_config['orbits'] = str(orbit_file)
45 N, N_day = int(round(T_obs * fs)), int(round(86400 * fs))
46 frequency_blocks = np.arange(
47     frequency_range[0], frequency_range[1] - freq_block_width * 1.e-10, freq_block_width,
48 )[:, None] + np.array([0., freq_block_width])
49 frequency_blocks[-1, 1] = frequency_range[1]
50 block_count = len(frequency_blocks)
51 block_indices = np.arange(block_count, dtype=np.int64)
52 subtraction_range = np.array([frequency_range[0], frequency_range[1]]) if
       subtraction_frequency_range is None else np.asarray(subtraction_frequency_range, dtype=float)
53 selected_block_indices = block_indices[
54     (frequency_blocks[:, 0] >= subtraction_range[0] - freq_block_width * 1.e-10)
55     & (frequency_blocks[:, 1] <= subtraction_range[1] + freq_block_width * 1.e-10)]
56 selected_block_count = len(selected_block_indices)
```

Lines 39–56 read the observation and its saved duration and sampling rate, then construct the indexed grid and select the requested blocks.

The grid is built once, before any global round. By default its lower edge is 10^{-4} Hz, its upper edge is 0.05 Hz, and the last block may be shorter than the configured width. At the default block width it contains 4,990 blocks with indices $0, \ldots, 4989$. Changing the subtraction interval leaves these edges and indices unchanged.

The subtraction interval is chosen to cover complete blocks. For a requested interval $[a, b]$, block j is selected when

$$f_j^{\mathrm{lo}} \geq a, \qquad f_j^{\mathrm{hi}} \leq b.$$

The small numerical tolerance only handles floating-point boundary rounding. The selected indices refer to the original `frequency_blocks` rows. Template evaluation and RJMCMC use each complete block's original bounds; there are no clipped intervals or separate `search_range` variables. The final grid upper edge is included once.

The caller creates the output's parent directory.

```
57 fields = dict(A=1, f=2, df=3, phi0=4, iota=5, longitude=6, latitude=7, psi_bme=8)
```

Line 57 maps each physical parameter to its column position in the numerical SNR table.

The dictionary provides the eight physical parameter column positions used for recovered waveforms and post-fit labels. Column 0 stores the ID, columns 1–8 store the physical parameters, and columns 9–10 store the absolute and relative SNRs. All DWDs use the same waveform and inference procedure.

The detector configuration is read directly from the observation; its filter, clock, padding and GPU settings are reused for fitted-waveform synthesis. Only the orbit-file location follows the separately specified companion path.

Parameter	Symbol	Catalogue field	Description
`A`	A	`Amplitude`	Gravitational-wave strain amplitude.
`f`	f	`Frequency`	Reference gravitational-wave frequency.
`df`	$\dot{f}$	`FrequencyDerivative`	Frequency derivative.
`phi0`	ϕ_0	`InitialPhase`	Initial waveform phase.
`iota`	ι	`Inclination`	Orbital inclination to the line of sight.
`longitude`	λ	`EclipticLongitude`	Ecliptic sky longitude.
`latitude`	β	`EclipticLatitude`	Ecliptic sky latitude.
`psi_bme`	ψ	`Polarization`	Polarization angle in the barycentric mean ecliptic frame.

4 Shared input, observation and result preparation

4.1 Step 3: read the inputs and prepare shared state (lines 58–73)

```
58 # Step 3. read the candidate bank and prepare shared state.
59 with h5py.File(snr_file, 'r') as saved:
60     template_dwd = saved['catalogue'][saved['catalogue'][:, 10] > SNR_r_threshold]
61 total_xyz_fd, fft_frequency = np.fft.rfft(total_xyz_td, axis=-1), np.fft.rfftfreq(N, d=1 / fs)
62 work = Path(mkdtemp(prefix='.dwd_all_', dir=output_file.parent)).resolve()
63 configuration = dict(duration=T_obs, rate=fs, frequency_range=np.array([frequency_range[0],
       frequency_range[1]]),
64     frequency_blocks_hz=frequency_blocks, block_indices=block_indices, frequency_block_width=
       freq_block_width,
65     subtraction_frequency_range=subtraction_range, selected_block_indices=selected_block_indices,
66     derivative_bounds=frequency_derivative_bounds,
67     chirp_coordinate='linear', f_threshold=F_threshold,
68     relative_snr_threshold=SNR_r_threshold, sample_number=np.asarray(sample_number),
69     minimum_sampler_capacity=maximum_sources)
70 with h5py.File(output_file, 'w') as saved:
71     _save_tree(saved.create_group('config'), configuration)
72     saved.create_group('blocks')
73     saved.create_group('rounds')
```

The result of `eric_snr.py` contains one numerical HDF5 dataset, `catalogue`, with shape $(29{,}000{,}000, 11)$. Its columns are `ID`, `Amplitude`, `Frequency`, `FrequencyDerivative`, `InitialPhase`, `Inclination`, `EclipticLongitude`, `EclipticLatitude`, `Polarization`, `SNR_absolute`, and `SNR_relative`, in that order. Column names are also saved in `settings/catalogue_columns`. All values use float64; the integer IDs 1–29,000,000 are represented exactly.

The loading block reads column 10 and directly selects complete rows with `SNR_relative` > `SNR_r_threshold`, currently 7. A relative SNR equal to 7 is excluded. The retained `template_dwd` has shape $(n_{\text{template}}, 11)$; the current result supplies 8,336 rows. The saved IDs are retained without renumbering or reconstructing them. The main pipeline passes selected rows of this same numerical table directly to the F-statistic and RJMCMC functions. Those functions read the required intrinsic columns internally without changing the input table. Templates are selected by the requested block frequency limits. The physical refinement bounds now come from the upper log-magnitude limit in `f_derivative_pn`: currently $[-10^{-10}, 10^{-10}]$ Hz/s. The sampler retains its signed linear derivative coordinate; the lower declared magnitude 10^{-25} is not a cutoff in this linear prior. These bounds do not generate additional templates.

The observation must contain finite `observation/total_xyz_td` with shape $(3, N)$, where $N = \text{round}(T_{\text{obs}} f_s)$. The duration and sampling rate are read from `observation` attributes `duration` and

`rate`; the subtraction script defines neither again. The original total observation is kept unchanged. Its unscaled real FFT, `total_xyz_fd`, and matching frequency axis are assigned together here. NumPy supplies the spectrum through `rfft` and the frequencies through `rfftfreq`. The frequency axis is reused throughout the frequency-block analysis.

The temporary directory `.dwd_all_*` holds response wrappers, post-fit label caches and sampler checkpoints. It is removed after successful finalization. The result is opened with mode `'w'`, so a rerun replaces an existing result. Configuration records the whole indexed grid as well as `selected_block_indices`; the selected subset is therefore recoverable from the result without guessing from populated block groups.

5 Global rounds and shared residual covariance

Steps 4–10 repeat for each global round. Steps 5–8 visit each active selected block; Steps 9–10 combine the accepted sources and decide whether another round is needed.

5.1 Step 4: start a round and estimate covariance (lines 74–95)

```
74 # Step 4. start a global round and estimate covariance.
75 round_number, last_accepted_round = 0, 0
76 print(f'Subtraction range {subtraction_range[0]:.8g}--{subtraction_range[1]:.8g} Hz: '
77       f'selected {selected_block_count} of {block_count} blocks, indices '
78       f'{selected_block_indices[0]}--{selected_block_indices[-1]} (zero-based).', flush=True)
79 with single_cpu():
80     while True:
81         round_number += 1
82         label = f'{round_number:02d}'
83         print(f'Global round {round_number}: visit {selected_block_count} selected frequency
     blocks.', flush=True)
84         # This complete Fourier snapshot is unchanged throughout the block loop.
85         round_residual_fd = total_xyz_fd if round_number == 1 else residual_xyz_fd
86         round_residual_fd.setflags(write=False)
87         round_digest = hashlib.sha256(round_residual_fd.tobytes()).hexdigest()
88         covariance_full = residual_covariance(round_residual_fd, T_obs, fs)
89         block_order = selected_block_indices
90         status = np.full(block_count, 'outside_subtraction_range', dtype='S32')
91         status[selected_block_indices] = 'pending'
92         with h5py.File(output_file, 'a') as saved:
93             group = saved['rounds'].create_group(label)
94             group.create_group('blocks')
95             group.attrs.update(complete=False, committed=False, residual_input_sha256=round_digest
     )
```

The first round analyzes `total_xyz_fd`. Later rounds analyze the residual left by the preceding subtraction round.

Every selected block fits the same full Fourier residual snapshot in a global round. The snapshot is made read-only and its digest is recorded before the block loop. All accepted candidates remain proposals until the selected pass is complete.

The covariance helper estimates the XYZ noise covariance from overlapping measured-data chunks. Every requested block enters the F-statistic search in index order; there is no spectral-excess or false-alarm pre-screen. Outside blocks retain the status `outside_subtraction_range`.

6 Block preparation and template-bank F-statistics

6.1 Step 5: prepare each selected block and evaluate its templates (lines 96–144)

```
96          # Step 5. evaluate each selected block against its recorded templates.
97          selected_counts = np.zeros(block_count, dtype=np.int64)
98          accepted_counts = np.zeros(block_count, dtype=np.int64)
99          proposals, proposal_doppler, proposal_coefficients, proposal_ids, proposal_blocks = [],
    [], [], [], []
100         for block_index in block_order:
101             key = f'{block_index:05d}'
102             block_range = frequency_blocks[block_index]
103             center = float(block_range.mean())
104             block_derivative_bounds = np.asarray(frequency_derivative_bounds, dtype=float)
105             padding = block_range[1] * 1.e-4 + max(abs(block_derivative_bounds)) * T_obs + 8 /
    T_obs
106             retained_bins = np.flatnonzero((fft_frequency >= block_range[0] - padding)
107                                            & (fft_frequency <= block_range[1] + padding) & (
    fft_frequency > 0))
108             center_bin = int(round(center * T_obs))
109             wide_bins = np.arange(max(1, min(center_bin - 2048, retained_bins[0] - 256)),
110                 min(len(fft_frequency), max(center_bin + 2049, retained_bins[-1] + 257)))
111             core_indices = retained_bins - wide_bins[0]
112             physical_frequency, wide_frequency = fft_frequency[retained_bins], fft_frequency[
    wide_bins]
113             if round_number == 1 and block_index == block_order[0]:
114                 builder, response_times, response_geometry = build_response(work, wide_frequency,
    T_obs, fs, orbit_file=orbit_file)
115                 builder_type = type(builder)
116             else:
117                 builder = builder_type(np.broadcast_to(np.eye(3), (len(wide_frequency), 3, 3)),
118                     np.zeros((3, len(wide_frequency)), complex), response_times, response_geometry
    , wide_frequency, T_obs, fs)
119             template_data = dict(derivative_bounds=block_derivative_bounds,
120                 retained_bins=retained_bins, wide_bins=wide_bins, core_indices=core_indices,
121                 block_index=block_index, frequency_range=block_range,
122                 physical_frequency_hz=physical_frequency)
123             with h5py.File(output_file, 'a') as saved:
124                 if key not in saved['blocks']:
125                     _save_tree(saved['blocks'].create_group(key), template_data)
126             covariance = covariance_full[retained_bins].copy()
127             inverse = np.linalg.inv(covariance)
128             covariance.setflags(write=False)
129             data = round_residual_fd[:, retained_bins]
130             target = (template_dwd[:, fields['f']] >= block_range[0]) & (
131                 (template_dwd[:, fields['f']] < block_range[1]) |
132                 ((block_index == block_count - 1) & (template_dwd[:, fields['f']] == block_range
    [1])))
133             search = template_f_statistic_search(data, covariance, builder, core_indices, T_obs,
    fs,
134                 template_dwd[target], f_threshold=F_threshold)
135             search['template_dwd_id'] = template_dwd[target, 0].astype(np.int64)
136             selected = search['template_dwd']
137             selected_counts[block_index] = len(selected)
138             block_data = dict(fixed_covariance=covariance,
139                 screening=dict(search=search, residual_input_xyz_fd=data))
140             block_attributes = dict(residual_input_sha256=round_digest,
141                 block_index=block_index,
142                 fixed_covariance_sha256=hashlib.sha256(covariance.tobytes()).hexdigest(),
143                 proposed_for_subtraction=False, accepted_for_subtraction=False)
144             status[block_index] = 'no_new_candidates'
```

This step creates the counts and proposal collections for the current block pass. Response geometry is obtained directly for the first selected block of the first round and then reused.

Each selected block retains measured bins beyond its carrier interval, with margin

$$\Delta f = f_{\text{upper}}\, 10^{-4} + \max|\dot{f}|\, T_{\text{obs}} + \frac{8}{T_{\text{obs}}}.$$

The wider response window has at least the nominal 4,097-bin support, clipped at FFT boundaries or enlarged if the retained bins require it. This support preserves sidebands and chirp power; it does not expand the allowed carrier interval.

The first selected block constructs moving-orbit response geometry. Later blocks reuse the same response times and geometry with their own physical frequency arrays. Covariance is copied from the measured full-band estimate and held read-only during the block's fit.

6.2 Evaluate the templates recorded in eric_snr

For each block, `template_f_statistic_search` receives rows of the unchanged 11-column `template_dwd` table whose carrier frequencies lie inside the complete selected block. It reads the $(f, \dot{f}, \lambda, \beta)$ columns internally and returns retained rows in the same table format for the RJMCMC call. It builds four basis waveforms for every template and analytically maximizes over the four amplitude coefficients:

$$\mathcal{F} = \frac{1}{2} x^{\mathsf{T}} M^{+} x.$$

There is no extra frequency, sky or chirp grid and no intrinsic-parameter optimizer before RJMCMC. Every supplied template is evaluated against the same block residual. Templates with $\mathcal{F} \geq F_{\text{threshold}}$ initialize RJMCMC, which refines the retained parameters. If no template passes, the search result is saved and the block has no sampler candidates.

7 Joint block inference and data-based acceptance

7.1 Step 6: run RJMCMC and assess the fitted sources (lines 145–204)

```
145             # Step 6. run RJMCMC and assess fitted sources.
146             if len(selected):
147                 print(f'  Block {key}, {block_range[0]:.8f}--{block_range[1]:.8f} Hz: fit {len(
    selected)} candidates.', flush=True)
148                 checkpoints = work / f'round_{label}_block_{key}'
149                 posterior = run_eryn_rjmcmc(data, covariance, selected, block_range,
150                     block_derivative_bounds, center, T_obs, fs, checkpoints,
151                     chain_count=1, inverse_temperatures=(1., .8, .6, .4),
152                     iterations=sum(sample_number), burn_in=sample_number[0],
153                     maximum_sources=max(maximum_sources, len(selected)), chirp_coordinate='linear'
    ,
154                     response_times=response_times, response_geometry=response_geometry,
    physical_frequency=physical_frequency)
155                 doppler, coefficients = posterior['doppler'], posterior['fit_result']['
    coefficients']
156                 new_bases = builder.build(doppler)[:, :, :, core_indices]
157                 new_sources = reconstruct_waveform(coefficients, new_bases)
158                 source_snr = np.sqrt(np.maximum(noise_inner_product(new_sources, new_sources,
    inverse, T_obs, fs), 0.))
159                 assessment_data = data.copy()
160                 # Fits share the round snapshot, but acceptance is conditional on
161                 # already proposed components to avoid subtracting a sideband twice.
162                 if proposal_doppler:
163                     points, amplitudes = np.concatenate(proposal_doppler), np.concatenate(
    proposal_coefficients)
```

```
                    nearby = abs(points[:, 0] - center) <= .5 * np.ptp(block_range) + 2 * padding
                    if np.any(nearby):
                        pending = reconstruct_waveform(amplitudes[nearby],
                            builder.build(points[nearby])[:, :, :, core_indices])
                        assessment_data -= pending.sum(axis=0)
                conditional_f = np.zeros(len(doppler))
                for i in range(len(doppler)):
                    conditional_data = assessment_data - new_sources.sum(axis=0) + new_sources[i]
                    conditional_f[i] = amplitude_f_statistic(conditional_data, new_bases[i:i + 1],
                        covariance, T_obs, fs)['two_f'][0] / 2
                owned = (doppler[:, 0] >= block_range[0]) & ((doppler[:, 0] < block_range[1])
                    | ((block_range[1] == frequency_range[1]) & (doppler[:, 0] == frequency_range
    [1])))
                duplicate = np.zeros(len(doppler), dtype=bool)
                previous_doppler = proposal_doppler if round_number == 1 else accepted_doppler +
    proposal_doppler
                previous_coefficients = proposal_coefficients if round_number == 1 else
    accepted_coefficients + proposal_coefficients
                if previous_doppler:
                    points, amplitudes = np.concatenate(previous_doppler), np.concatenate(
    previous_coefficients)
                    nearby = abs(points[:, 0] - center) <= .5 * np.ptp(block_range) + 2 * padding
                    if np.any(nearby):
                        previous = reconstruct_waveform(amplitudes[nearby],
                            builder.build(points[nearby])[:, :, :, core_indices])
                        overlap = noise_inner_product(new_sources[:, None], previous[None, :],
    inverse, T_obs, fs)
                        norm = np.sqrt(np.maximum(noise_inner_product(new_sources, new_sources,
    inverse, T_obs, fs)[:, None]
                            * noise_inner_product(previous, previous, inverse, T_obs, fs)[None,
    :], 1.e-300))
                        duplicate |= np.any(overlap / norm > .9, axis=1)
                retained = owned & ~duplicate & np.isfinite(source_snr) & (source_snr >
    SNR_r_threshold) & (conditional_f >= F_threshold)
                # Deduplicate the fitted components using measured waveform overlap.
                for i in np.argsort(source_snr)[::-1]:
                    if not retained[i]:
                        continue
                    for j in np.flatnonzero(retained):
                        if j == i or source_snr[j] > source_snr[i] or (source_snr[j] == source_snr
    [i] and j < i):
                            continue
                        overlap = noise_inner_product(new_sources[i], new_sources[j], inverse,
    T_obs, fs)
                        if overlap / max(source_snr[i] * source_snr[j], 1.e-300) > .9:
                            retained[j], duplicate[j] = False, True
                retained_sum = new_sources[retained].sum(axis=0)
                improvement = float(noise_inner_product(assessment_data, assessment_data, inverse,
     T_obs, fs)
                    - noise_inner_product(assessment_data - retained_sum, assessment_data -
    retained_sum, inverse, T_obs, fs))
                accepted = np.any(retained) and improvement > 0
                status[block_index] = 'proposed' if accepted else 'fit_rejected_by_data'
```

Each active seeded block invokes Eryn once against the frozen residual and fixed covariance. The default effort is 500 burn-in plus 1,500 production cycles, with one walker and inverse temperatures $(1, 0.8, 0.6, 0.4)$. The controller explicitly requests the signed linear chirp coordinate. The source capacity is at least the configured capacity and the seed count.

Fitting still sees the original round snapshot. Acceptance uses a separate assessment array: nearby signals proposed earlier in this round are removed from that array before judging a later proposal. This prevents neighboring blocks from claiming the same measured feature twice without changing the sampler's input mid-round.

For source i, the conditional F statistic evaluates the remaining assessment data with the other fitted components removed and component i retained. An accepted component must have an owned carrier, finite fitted SNR strictly above 7, conditional $\mathcal{F} \geq 16$, and no measured waveform duplicate. Signed normalized waveform overlap greater than 0.9 detects duplicates against previously accepted and already proposed sources; stronger members of an overlapping fitted pair are retained.

The retained sum must also reduce the noise-weighted assessment residual norm:

$$\Delta = (y_{\text{assess}} \,|\, y_{\text{assess}})_C - (y_{\text{assess}} - h_{\text{retained}} \,|\, y_{\text{assess}} - h_{\text{retained}})_C > 0.$$

These decisions depend on measured data and fitted waveforms. They do not require the recovered source to appear in a catalogue.

7.2 Step 7: associate catalogue labels after inference (lines 205–249)

```
205                 # Step 7. associate recovered sources with catalogue labels.
206                 # Catalogue reads happen after inference and have no acceptance role.
207                 if round_number == 1 and np.count_nonzero(selected_counts) == 1:
208                     with h5py.File(snr_file, 'r') as saved:
209                         truth_frequency = saved['catalogue'][:, fields['f']]
210                         chirp = saved['catalogue'][:, fields['df']]
211                     maximum_drift = float(np.max(abs(chirp) * T_obs + abs(truth_frequency) * 1.e-4
     + 8 / T_obs))
212                     truth_order = np.argsort(truth_frequency, kind='stable')
213                     truth_frequency = truth_frequency[truth_order]
214                 # Truth is consulted only after fitting, and cached on disk for this block.
215                 truth_cache = work / f'truth_{key}.h5'
216                 if truth_cache.exists():
217                     with h5py.File(truth_cache, 'r') as saved:
218                         truth_ids = saved['dwd_id'][:]
219                         truth_waveforms = saved['source_xyz_fd'][:]
220                 else:
221                     truth_low, truth_high = block_range + np.array([-2304., 2304.]) / T_obs
222                     first, last = np.searchsorted(truth_frequency, [truth_low - maximum_drift,
    truth_high + maximum_drift])
223                     nearby_indices = np.sort(truth_order[first:last])
224                     with h5py.File(snr_file, 'r') as saved:
225                         records = saved['catalogue'][nearby_indices]
226                     frequency = records[:, fields['f']]
227                     end = frequency + records[:, fields['df']] * T_obs
228                     margin = np.maximum(frequency, end) * 1.e-4 + 8 / T_obs
229                     nearby = (np.minimum(frequency, end) - margin <= truth_high) & (np.maximum(
    frequency, end) + margin >= truth_low)
230                     truth_rows = records[nearby]
231                     truth_ids = truth_rows[:, 0].astype(np.int64)
232                     truth_doppler = truth_rows[:, [fields[name] for name in ('f', 'df', 'longitude
    ', 'latitude')]]
233                     cos_iota = np.cos(truth_rows[:, fields['iota']])
234                     truth_coefficients = physical_amplitude_coordinates(truth_rows[:, fields['A']]
     * (1 + cos_iota**2),
235                         2 * truth_rows[:, fields['A']] * cos_iota, truth_rows[:, fields['psi_bme'
    ]], truth_rows[:, fields['phi0']]) / 1.e-22
236                     truth_waveforms = np.empty((len(truth_ids), 3, len(retained_bins)), dtype=
    complex)
237                     for first in range(0, len(truth_ids), 32):
238                         truth_waveforms[first:first + 32] = reconstruct_waveform(
    truth_coefficients[first:first + 32],
239                             builder.build(truth_doppler[first:first + 32])[:, :, :, core_indices])
240                     with h5py.File(truth_cache, 'w') as saved:
241                         saved.create_dataset('dwd_id', data=truth_ids)
242                         saved.create_dataset('source_xyz_fd', data=truth_waveforms)
243                 truth_association = assign_bank_ids(new_sources, truth_waveforms, truth_ids,
    inverse, T_obs, fs, minimum_overlap=.9)
```

```
244                 matched = truth_association['matched_dwd_id'].copy()
245                 physical = physical_amplitudes(coefficients)
246                 parameters = np.column_stack((physical['A'], doppler[:, 0], doppler[:, 1],
    physical['phi0'],
247                     physical['iota'], doppler[:, 2], doppler[:, 3], physical['psi_bme']))
248                 block_data.update(catalogue_association=dict(matched_dwd_id=matched,
249                     best_overlap=truth_association['best_overlap'], scope='Post-fit labels only;
    unmatched fits may be subtracted.'),
```

The catalogue frequency lookup and its maximum frequency margin are calculated here, when the first fitted block in the first round reaches Step 7. Both are reused for later blocks and rounds.

Only after the fitted-source assessment does the controller scan the SNR catalogue's frequencies and derivatives. It constructs a sorted index once, then reads nearby rows directly from the same numerical `catalogue` dataset for each block. The saved ID and eight physical parameter columns supply the labels and reference waveforms.

The code names this temporary label view `truth_*`. Here it provides post-fit labels only. Its waveforms do not seed the search, supply a prior, control acceptance or stop the global loop. Local label waveforms are cached for later rounds. A source absent from the file can still be accepted.

The accepted physical row is reconstructed from the fitted amplitude coefficients and Doppler parameters, in the order $(A, f, \dot{f}, \phi_0, \iota, \lambda, \beta, \psi)$. The four-amplitude inverse has physical angle degeneracies, so this row represents an equivalent waveform rather than a uniquely identifiable set of angles.

7.3 Step 8: allocate source IDs and record the fits (lines 250–292)

```
250                 # Step 8. record the accepted fits.
251                 acceptance=dict(source_snr=source_snr, conditional_f=conditional_f,
252                     owned_by_block=owned, duplicate_previous_or_this_round=duplicate,
253                     retained=retained, likelihood_improvement=improvement),
254                 recovery=dict(doppler=doppler, coefficients=coefficients, parameters=
    parameters, source_xyz_fd=new_sources),
255                 rjmcmc={name: value for name, value in posterior.items() if name not in ('
    bases', 'fit_result', 'checkpoint_file')})
256             block_attributes.update(proposed_for_subtraction=accepted,
257                 source_xyz_fd_scope='Frequency-domain fitting and post-fit association only.')
258             if accepted:
259                 retained_ids = matched[retained].copy()
260                 if round_number == 1 and not np.any(accepted_counts):
261                     next_unknown_id = -1
262                 if round_number == 1:
263                     used_labels = np.concatenate(proposal_ids) if proposal_ids else np.empty
    (0, dtype=np.int64)
264                 else:
265                     used_labels = np.concatenate(accepted_ids + proposal_ids)
266                 for i in range(len(retained_ids)):
267                     if retained_ids[i] < 0 or retained_ids[i] in used_labels or retained_ids[i
    ] in retained_ids[:i]:
268                         retained_ids[i] = next_unknown_id
269                         next_unknown_id -= 1
270                 proposals.append(parameters[retained])
271                 proposal_doppler.append(doppler[retained])
272                 proposal_coefficients.append(coefficients[retained])
273                 proposal_ids.append(retained_ids)
274                 proposal_blocks.append(np.full(np.count_nonzero(retained), block_index, dtype=
    np.int64))
275                 accepted_counts[block_index] = np.count_nonzero(retained)
276             snapshots = {}
277             for path in sorted(checkpoints.glob('*.h5')):
278                 with h5py.File(path, 'r') as saved:
279                     snapshots[path.stem] = read_checkpoint(saved)
```

```
280                 with h5py.File(output_file, 'a') as saved:
281                     group = saved[f'rounds/{label}/blocks'].create_group(key)
282                     _save_tree(group, block_data)
283                     group.attrs.update(block_attributes)
284                     checkpoint_group = group.create_group('checkpoints')
285                     for name, snapshot in snapshots.items():
286                         write_checkpoint(checkpoint_group.create_group(name), snapshot)
287                     group['rjmcmc'].attrs['checkpoint_group'] = f'/rounds/{label}/blocks/{key}/
    checkpoints'
288             else:
289                 with h5py.File(output_file, 'a') as saved:
290                     group = saved[f'rounds/{label}/blocks'].create_group(key)
291                     _save_tree(group, block_data)
292                     group.attrs.update(block_attributes)
```

The negative-ID counter starts when the first accepted fit needs label allocation. The first round compares labels against its current proposals; later rounds also include already accepted IDs.

Waveform association supplies an available positive row ID when a match exists. Unmatched components, already used labels and repeated labels within a fit receive fresh negative IDs. Global block indices are stored separately from source IDs. Identity labels therefore cannot force duplicate subtraction.

Per-block records retain the search, fixed covariance, posterior output, recovered FD components, source SNR, conditional F, ownership and duplicate decisions, and the retained mask. Sampler HDF5 checkpoints are copied into the result file before successful temporary-directory cleanup. The embedded checkpoint group is recorded in the sampler attributes.

8 Combined time-domain subtraction and finalization

8.1 Step 9: subtract the combined time-domain signal (lines 293–328)

```
293         # Step 9. subtract the combined time-domain TDI signal.
294         if round_number == 1:
295             accepted_parameters, accepted_doppler, accepted_coefficients, accepted_ids,
    accepted_blocks = [], [], [], [], []
296             accumulated_xyz_td = np.zeros_like(total_xyz_td)
297             residual_xyz_td, residual_xyz_fd = total_xyz_td.copy(), total_xyz_fd
298         # Only here may the residual change: one waveform/TDI sum for all accepted blocks.
299         if proposals:
300             parameters = np.concatenate(proposals)
301             print(f'  Subtract {len(parameters)} fitted DWDs from {np.count_nonzero(
    accepted_counts)} blocks together.', flush=True)
302             links = DWD_simulation_gpu(parameters, detector_config)
303             response_model = lisagwresponse.GalacticBinary(A=1., f=.003, ra=0., dec=0.,
304                 orbits=str(orbit_file), orbit_interp_order=5, shift_sun='never')
305             links_file = work / 'recovered_links.h5'
306             with h5py.File(links_file, 'w') as saved:
307                 response_model.init_response_file(saved, detector_config['dt'], len(links), -2 *
    N_day / fs, 'tcb')
308                 saved['tcb/y'][:] = links
309             del links
310             instrument = lisainstrument.Instrument(
311                 **{name: value for name, value in detector_config.items()
312                    if name not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', '
    include_padding')},
313                 gws=str(links_file), size=N + 2 * N_day, t0=-N_day / fs)
314             instrument.disable_all_noises()
315             instrument.disable_dopplers()
316             data = pytdi.Data.from_instrument(instrument.export_numpy_full())
```

```
317             new_xyz_td = tdi_gpu(data.measurements, data.args, keep=slice(N_day, N_day + N)) /
    instrument.central_freq
318             accumulated_xyz_td += new_xyz_td
319             residual_xyz_td = total_xyz_td - accumulated_xyz_td
320             residual_xyz_fd = np.fft.rfft(residual_xyz_td, axis=-1)
321             del data, instrument, response_model, new_xyz_td
322             accepted_parameters.append(parameters)
323             accepted_doppler.extend(proposal_doppler)
324             accepted_coefficients.extend(proposal_coefficients)
325             accepted_ids.extend(proposal_ids)
326             accepted_blocks.extend(proposal_blocks)
327             last_accepted_round = round_number
328             status[status == b'proposed'] = 'accepted'
```

Accepted-fit collections, accumulation and residual arrays are initialized here on the first round, when the block pass reaches subtraction. If no sources are accepted, the residual remains the original total observation and the recovered signal remains zero.

The residual changes only after every selected block has been assessed. Accepted physical parameter rows from all those blocks are concatenated and passed to `DWD_simulation_gpu` once. Its summed six-link response is wrapped in the native LISA response-file format and converted by a noise-free native instrument. Full measurement dictionaries are then passed to `tdi_gpu`.

The instrument filter and moving-arm delays follow the simulation input's configuration. One padding day is removed from each end of the instrument/TDI stream. Division by the central optical frequency produces the fractional- frequency XYZ convention of the stored observation.

Let $D(t)$ denote the original three-channel observation, $H_{r-1}(t)$ the previously accepted waveform and $\Delta H_r(t)$ this round's new accepted sum. The implemented update is

$$H_r(t) = H_{r-1}(t) + \Delta H_r(t), \tag{3}$$

$$R_r(t) = D(t) - H_r(t), \tag{4}$$

$$d_r = \mathrm{rfft}(R_r). \tag{5}$$

Previously accepted source parameters stay frozen. The next round uses the fresh FFT of the new shared residual. Models retain their physical sidebands even when their carrier frequencies were selected in a restricted interval.

8.2 Step 10: record the round and decide whether to continue (lines 329–346)

```
329         # Step 10. record the round and decide whether to continue.
330         recovered_ids = np.concatenate(accepted_ids) if accepted_ids else np.empty(0, dtype=np.
    int64)
331         round_summary = dict(block_status=status, selected_count=selected_counts, accepted_count=
    accepted_counts,
332             recovered_dwd_id=recovered_ids)
333         with h5py.File(output_file, 'a') as saved:
334             group = saved[f'rounds/{label}']
335             _save_tree(group, round_summary)
336             group.attrs.update(complete=True, committed=bool(proposals), visited_block_count=
    selected_block_count,
337                 selected_block_count=selected_block_count,
338                 accepted_block_count=int(np.count_nonzero(accepted_counts)),
339                 residual_output_sha256=hashlib.sha256(residual_xyz_fd.tobytes()).hexdigest())
340             for block_index in np.flatnonzero(accepted_counts):
341                 saved[f'rounds/{label}/blocks/{block_index:05d}'].attrs.update(
    accepted_for_subtraction=True,
342                     subtraction_backend='DWD_simulation_gpu + native Instrument + tdi_gpu')
343             saved.attrs.update(last_completed_round=round_number, last_accepted_round=
    last_accepted_round)
344         if not proposals:
```

```
345             stopping_reason = 'no_new_candidates' if not np.any(selected_counts) else '
    no_accepted_blocks'
346             break
```

The recovered-ID array is assembled here from accepted fit records, when recording the round result.

Round statuses are stored over the whole indexed grid. Selected blocks can lack F-statistic candidates, have a rejected fit, or commit accepted components. Other indices remain outside the subtraction range.

If no block proposes an accepted waveform, the controller stops with `no_new_candidates` or `no_accepted_blocks`. There is no maximum round count, catalogue-exhaustion or ghost-fraction stopping rule.

8.3 Step 11: save the final results and finish (lines 347–365)

```
347 # Step 11. save the final results.
348 parameters = np.concatenate(accepted_parameters) if accepted_parameters else np.empty((0, 8))
349 ids = np.concatenate(accepted_ids) if accepted_ids else np.empty(0, dtype=np.int64)
350 final_data = dict(signal_xyz_td=accumulated_xyz_td, residual_xyz_td=residual_xyz_td,
351     signal_xyz_fd=np.fft.rfft(accumulated_xyz_td, axis=-1), residual_xyz_fd=residual_xyz_fd,
352     parameters=parameters, dwd_id=ids,
353     doppler=np.concatenate(accepted_doppler) if accepted_doppler else np.empty((0, 4)),
354     coefficients=np.concatenate(accepted_coefficients) if accepted_coefficients else np.empty((0,
        4)),
355     block_index=np.concatenate(accepted_blocks) if accepted_blocks else np.empty(0, dtype=np.int64
        ))
356 with h5py.File(output_file, 'a') as saved:
357     _save_tree(saved.create_group('final_subtraction'), final_data)
358     saved['final_subtraction'].attrs['accepted_round'] = last_accepted_round
359     saved.attrs.update(complete=True, single_file_complete=True, stopping_reason=stopping_reason,
360         stopping_condition_met=True, iteration_stable=False)
361 shutil.rmtree(work)
362 print(f'Saved {output_file}: {round_number} global rounds, {len(parameters)} accepted components;
        {stopping_reason}.', flush=True)
363 pipeline_result = dict(output_file=str(output_file), rounds=round_number, source_count=len(
        parameters),
364             selected_block_indices=selected_block_indices.tolist(), total_block_count=block_count,
365             last_accepted_round=last_accepted_round, stopping_reason=stopping_reason)
```

The final result contains the accumulated TD and FD signal, shared residual, accepted physical parameters, four-amplitude coefficients, Doppler points, source IDs and global block indices. The zero-source case writes empty parameter arrays and retains the original observation exactly.

Set `subtraction_frequency_range` in Step 2 to `(f_min,f_max)` in Hz. The final `pipeline_result` dictionary records `selected_block_indices`. Root completion is written after final arrays; successful execution removes the workspace. The implementation does not provide transactional replacement of a previous result or an automatic resume from a partially written result.

9 Mathematical model and normalization

9.1 Raw FFT and the correlated XYZ covariance

For a real channel $x_c[m]$, the forward coefficients are

$$d_{c,k} = \sum_{m=0}^{N-1} x_c[m] \exp(-2\pi i k m/N).$$

There is no $1/N$ or sample-spacing factor in this forward FFT. For interior positive-frequency bins, the one-sided CSD convention is

$$\mathbb{E}[n_k n_k^\dagger] = \frac{T_{\rm obs} f_s^2}{2} C_k, \qquad P_k = \frac{2}{T_{\rm obs} f_s^2} d_k d_k^\dagger. \tag{6}$$

The periodogram of a single bin is rank one. Neighboring-frequency smoothing produces the full covariance used for the likelihood.

The inspected inner product retains complex XYZ cross terms:

$$(a \,|\, b)_C = \frac{4}{T_{\rm obs} f_s^2} \,\mathrm{Re} \sum_k a_k^\dagger C_k^{-1} b_k. \tag{7}$$

If $C_k = L_k L_k^\dagger$, Cholesky whitening of each channel vector followed by concatenation of real and imaginary parts gives an ordinary real dot product equal to Equation (7). This is the convention used by the GPU amplitude integral.

DC is excluded from the covariance estimate and retained fitting bins. Nyquist is included in the full default grid, while the covariance and inner-product formulas do not introduce a special real-endpoint weight. Interior-bin normalization is the verified reference; endpoint handling remains a specific check for searches near Nyquist, as discussed in Section 13.

9.2 Measured covariance smoothing and peak downweighting

The smoothing helper uses a normalized cosine kernel with half-width 256 bins, giving 513 support entries:

$$K_\ell \propto \tfrac{1}{2}[1 + \cos(\pi \ell/256)], \qquad |\ell| \le 256, \qquad \sum_\ell K_\ell = 1.$$

This smooths the covariance in frequency, not the observation in time. With peak downweighting, it iterates at most 12 times. For $s_k = \mathrm{Re}\,\mathrm{tr}(\widehat{C}_k^{-1} P_k)$, the weight is $w_k = \max(1 - s_k/15, 0)^2$. The weighted local mean is divided by its kernel weight sum and the factor $c = 0.8503365527522067$.

For ideal proper-complex Gaussian XYZ noise, $S \sim \mathrm{Gamma}(3, 1)$. Writing $w(S) = (1 - S/15)^2 \mathbf{1}_{S<15}$, the scale correction is

$$c = \frac{\mathbb{E}[S\, w(S)]}{3\,\mathbb{E}[w(S)]}.$$

Its ideal-model derivation does not establish unbiasedness for coherent foreground, finite windows or covariance estimated from the same samples. The helper symmetrizes its estimate and adds a small trace-scaled diagonal term. The full-band screen adds a further positive scale floor, or an $I_3\, 10^{-100}$ fallback for an exactly zero chunk. Per-fit covariance is stored, but numerical health and estimator convergence are not independently certified by a completion flag.

9.3 Four linear amplitudes and the F projection

At a fixed Doppler point $\theta = (f, \dot{f}, \lambda, \beta)$, one source is

$$h(\theta, a) = \sum_{\mu=1}^{4} a_\mu H_\mu(\theta).$$

The physical convention is $A_+ = A(1 + \cos^2 \iota)$ and $A_\times = 2A \cos \iota$. With $c_\psi = \cos 2\psi$, $s_\psi = \sin 2\psi$, $c_\phi = \cos \phi_0$, $s_\phi = \sin \phi_0$,

$$\mathcal{A}_1 = A_+ c_\psi c_\phi + A_\times s_\psi s_\phi, \tag{8}$$
$$\mathcal{A}_2 = A_+ s_\psi c_\phi - A_\times c_\psi s_\phi, \tag{9}$$
$$\mathcal{A}_3 = -A_+ c_\psi s_\phi + A_\times s_\psi c_\phi, \tag{10}$$
$$\mathcal{A}_4 = -A_+ s_\psi s_\phi - A_\times c_\psi c_\phi. \tag{11}$$

The coefficient arrays use $a_\mu = \mathcal{A}_\mu / 10^{-22}$, with the corresponding 10^{-22} basis scale.

Define $x_\mu = (H_\mu \,|\, d)_C$ and $M_{\mu\nu} = (H_\mu \,|\, H_\nu)_C$. The amplitude log likelihood ratio is $a^\mathsf{T} x - a^\mathsf{T} M a/2$. Its maximized value gives

$$2\mathcal{F} = x^\mathsf{T} M^+ x,$$

where the current vectorized helper uses a Hermitian pseudoinverse with `rcond=1e-12`. This admits rank-deficient bases as a projection. Numerical agreement with the existing full-rank projection is checked separately from selection of the recorded bank templates.

The SNR used for final acceptance is the fitted waveform norm

$$\rho_i = \sqrt{\max\{(h_i^{\text{fitted}} \,|\, h_i^{\text{fitted}})_C, 0\}}.$$

It uses fitted conditional-mean amplitudes and the measured effective covariance. It is distinct from a stored catalogue SNR and from the conditional F statistic after other fitted or pending components have been removed.

9.4 Joint amplitude marginalization and the zero-source model

For n sources, stack $4n$ basis vectors, including their cross-source Gram terms. The normalized amplitude prior is $a \sim \mathcal{N}(0, \sigma^2 I)$, with $\sigma = 10$ in the scaled coefficient units. Let $Q = M + \sigma^{-2} I$. The amplitude-integrated likelihood is

$$\log p(d \mid \theta, n, C) = -\tfrac{1}{2} (d \,|\, d)_C + \tfrac{1}{2} x^\mathsf{T} Q^{-1} x - \tfrac{1}{2} \log\det Q - 4n \log \sigma. \tag{12}$$

The determinant and prior normalization penalize additional amplitude dimensions. For $n = 0$, the amplitude terms vanish.

The GPU integral pads to four times the configured capacity, 196 rows at the default capacity of 49. Inactive zero-basis blocks contribute $Q = \sigma^{-2} I$, whose determinant and prior terms cancel. Stored amplitude draws use the conditional Gaussian covariance Q^{-1}; the subtracted point estimate uses its conditional mean $Q^{-1} x$.

Noise covariance and the residual are fixed within each block fit. Their noise-normalization constant can be omitted from within-fit ratios, but likelihood traces from different residuals or covariances are not directly comparable evidences and should not be pooled as one posterior.

10 RJMCMC and waveform helper contracts

10.1 Signed sampled coordinates and count prior

The global controller explicitly sets `chirp_coordinate='linear'`:

$$q_f = f_c T_{\rm obs} \ln(f/f_c), \qquad q_{\dot f} = \dot f\, T_{\rm obs}^2, \tag{13}$$

$$q_\lambda = \lambda \bmod 2\pi, \qquad q_\beta = \sin\beta. \tag{14}$$

The inverse is $f = f_c \exp[q_f/(f_c T_{\rm obs})]$, $\dot f = q_{\dot f}/T_{\rm obs}^2$, and $\beta = \arcsin q_\beta$. The helper's general default log-chirp option does not describe this caller. Its rectangular prior implies a $1/f$ frequency density, uniform signed derivative, uniform longitude and isotropic sky latitude.

For capacity $M_{\max}$, the exchangeable active-slot prior has

$$\log p(q, \text{active}) = -n \log V - \log \binom{M_{\max}}{n}, \quad 0 \le n \le M_{\max}.$$

Summing masks and integrating active coordinates gives equal integrated count weight for $n = 0, \ldots, M_{\max}$. The class name `CataloguePrior` refers to this variable-count parameter set, not to an external source list. The sampled count concerns additional sources in this block and round; the cumulative recovered count can exceed 49 across blocks and rounds.

10.2 Proposals, temperatures and the selected point estimate

Within-model Gaussian proposals move one active source in the encoded coordinates, with standard deviations $(0.025, 0.3, 0.015, 0.015)$ and temperature scaling. The derivative step is linear here, rather than a multiplicative log-chirp step. Birth proposals mix 90% candidate-centered boxes with 10% full-prior draws; when no centers are supplied, the full prior is used. The custom RJ factor compensates exchangeable slot selection. The code and installed Eryn behavior jointly determine balance and mixing.

The four temperatures form one ladder, not four independent cold chains. After burn-in, the helper chooses the modal cold source count, then the highest marginal-likelihood draw at that count. Conditional-mean amplitudes at that Doppler point generate the fitted waveform. This is a selected point estimate; it does not average over source count, physical parameters or subtraction uncertainty.

Stored traces are useful diagnostics, but this execution does not assess independent-chain agreement, effective sample size or robustness to longer sampling. A finite stopping condition or constant count trace is insufficient to claim convergence.

10.3 DWD parameter-table GPU interface

`DWD_simulation_gpu` in `eric_generate_waveform.py` accepts either an $(n, 8)$ float table or a structured physical-parameter table. Additional ID/SNR fields are ignored for synthesis. Each supplied row contributes once, including repeated parameter sets; no LDC lookup is needed.

It returns summed float64 dimensionless link fractional-frequency data in native link order $(12, 23, 31, 13, 32, 21)$. With padding enabled, the shape is $(N + 4N_{\rm day}, 6)$, covering two days at each side. A noise-free instrument then produces measurements with $N + 2N_{\rm day}$ samples. The final TDI crop returns the central $(3, N)$ observation. The observation producer and subtraction controller use this same sequence.

10.4 Native-expression GPU TDI interface

`tdi_gpu` defaults to `combination='XYZ2'`. It accepts six science-link measurement streams, or the full native measurement dictionary. The pipelines use dictionaries containing science, test-mass and reference beatnotes so that the standard instrumental-noise combinations are retained. Its input must already include instrument filtering.

The detector configuration can provide an orbit file or precomputed native `Data.args` containing moving-arm delays and delay derivatives. The implementation uses the installed PyTDI Michelson expressions with GPU Lagrange interpolation and preserves native orders 5/31 and float64 arithmetic. TDI itself adds no noise, FFT or file output. The output shape is $(\ldots, 3, N_{\mathrm{kept}})$ in X/Y/Z order and in the input's units. The current-controller native comparison is recorded in Section 12.

11 Array contracts and HDF5 interpretation

11.1 Array shapes and time-domain units

Array	Shape	Scope
Original, accumulated signal, residual XYZ TD	$(3, N)$	Whole observation.
Shared XYZ FD	$(3, \lfloor N/2 \rfloor + 1)$	Raw real FFT.
Block measured XYZ FD	$(3, K)$	Retained bins including sidebands.
Fixed block covariance	$(K, 3, 3)$	Full complex XYZ CSD.
Source basis set	$(n, 4, 3, K)$	Four amplitude bases per intrinsic point.
Fitted coefficients	$(n, 4)$	Amplitudes scaled by 10^{-22}.
Fitted Doppler rows	$(n, 4)$	$f, \dot{f}, \lambda, \beta$.
Recovered physical rows	$(n, 8)$	Physical waveform parameters.
Summed padded GW links	$(N + 4N_{\mathrm{day}}, 6)$	Before instrument filtering.
Padded native measurements	$N + 2N_{\mathrm{day}}$ per beatnote	Before central TDI crop.
Source IDs, global block IDs	$(n,)$	Separate identity fields.

At 360 days and 0.1 Hz, N = 3,110,400 and N_{day} = 8,640. Thus the summed padded links have 3,144,960 rows, native measurements have 3,127,680 samples, and final XYZ has 3,110,400 samples per channel. The physical input amplitude is strain; the returned links and normalized XYZ observation are dimensionless fractional-frequency responses. Matching dimensions and units are both required before addition or subtraction.

11.2 Root groups and per-round records

`config/` Frequency-block pairs and indices, subtraction range, selected indices, thresholds, derivative bounds, sampling effort and resource limits.

`blocks/<index>/` Static selected-block windows, retained bins, physical frequencies and the original full block interval. Outside blocks need not have a group; their indices remain in `config/` and the round status arrays.

`rounds/<round>/` Whole-grid statuses and source counts, residual input/output digests and commit state.

`rounds/<round>/blocks/<index>/` Fixed covariance, search result, fitted posterior, recovery rows, post-fit labels, acceptance masks and embedded sampler checkpoints.

final_subtraction/ Full TD/FD signal and residual, cumulative physical rows, Doppler points, amplitudes, source IDs and global block IDs.

Root `complete=True` means final publication finished. Round `complete=True` means that attempt was recorded; `committed=True` means it updated the residual. `stopping_condition_met` excludes resource-capped termination. `iteration_stable=False` records publication without a stability claim. The per-block sampler record separately states whether independent-chain convergence was assessed; this information is not a root attribute.

11.3 A direct result reader

```
with h5py.File('data/dwd_subtraction_all.h5', 'r') as saved:
    complete = saved.attrs['complete']
    grid_indices = saved['config/block_indices'][:]
    selected_indices = saved['config/selected_block_indices'][:]
    requested_range = saved['config/subtraction_frequency_range'][:]
    parameters = saved['final_subtraction/parameters'][:]
    source_ids = saved['final_subtraction/dwd_id'][:]
    source_blocks = saved['final_subtraction/block_index'][:]
    residual = saved['final_subtraction/residual_xyz_td'][:]
```

Calculations on these arrays belong outside the HDF5 loading scope. A catalogue label is post-fit metadata; a negative label is not a rejection. The original observation remains a separate input, while checkpoint arrays needed to inspect completed fits are embedded in the result.

12 Verification and measured scope

12.1 Current indexed-range controller

Runtime validation guards and explicit exceptions have been removed from the controller and its reviewed helpers. The numerical evidence below records the earlier implementation; valid-input regression checks accompany this revision.

The current primary-source SHA-256 is `6e34519a839f511d13ac10e370375f5d891bd4bc3710f424a20eaafd143c9db6`. The current revision passed 43 server regressions: 19 sequential-controller checks, five F-statistic/covariance checks, 17 observation-file checks, and two TDI/subtraction checks. These include strict selection on saved relative SNR, preservation of saved IDs and all 11 numerical table columns, reuse of saved detector settings, and the native Kaiser-filter coefficient/group-delay round trip. The numerical 11-column, named-table and four-coordinate inputs produce identical F-statistics, amplitude coefficients and encoded sampler coordinates in the regression fixtures; the input tables remain unchanged.

The existing SNR result was migrated to a numerical `catalogue` dataset with shape $(29{,}000{,}000, 11)$. A streaming comparison of every saved value against the original catalogue rows gave identical SHA-256 digests, preserving the parameters, both SNRs and original merged-row IDs. The revised `eric_snr.py` writes this table during its final relative-SNR pass; a fixture checks its actual export statements and saved column names. The current subtraction reader loads 8,336 qualifying rows without reconstructing IDs.

A ten-source, one-day moving-orbit GPU smoke compared the new 11-column table input with the existing eight-parameter input to `DWD_simulation_gpu`. Both returned shape $(8{,}640, 6)$ at 0.1 Hz, with maximum absolute difference and relative L^2 difference zero. The input table was unchanged. This verifies the table-input route; it is separate from the native LISA comparisons below. The records are `/home/genshin/dwd_injection/tmp/review_sync_20261012.gUWiIy/merged_snr_table_verification.json`, `/home/genshin/dwd_injection/tmp/review_sync_20261012.gUWiIy/merged_t`

able_integration.json, and /home/genshin/dwd_injection/tmp/review_sync_20261012.gUWiIy/merged_table_gpu.json.

The updated simulation was rerun on GPU1's A100 for all 29,000,000 DWDs over 360 days at 0.1 Hz, completing in about 19.5 minutes. The regenerated observation stores duration, rate, and detector_config. Its total, signal and noise TDI arrays each have shape $(3, 3{,}110{,}400)$ and contain finite values. The relative L^2 difference between total and signal plus noise is 1.93×10^{-14}. The companion orbit agrees with the embedded orbit. The run record is /home/genshin/dwd_injection/tmp/detector_config_rerun_20261012.55wr6r_k/verification.json.

The following remote smoke and timing results describe the earlier blind-search implementation and do not establish recovery for the current template bank.

The A100 integration smoke reused a 30-day 0.1 Hz native LISA observation with one unlisted DWD and continuous ACC+OMS noise, moving unequal arms, and an empty SNR catalogue. Its 32 burn-in plus 64 production cycles test integration rather than convergence.

Requested range (Hz)	Global indices	Observed result
0.002–0.00203	199, 200, 201, 202	No sampler or TDI call; original observation preserved exactly.
0.0030005–0.003008	300	One accepted source, ID −1, one sampler and one TDI call; only this block visited in both global rounds.

The fitted carrier was 0.0030031746626394866 Hz, within the requested interval. The recovered signal's relative L^2 difference from the native injected signal was 0.0182088121. This is a fitted-waveform error, not a GPU interpolation error. GPU TDI on the recovered measurements was bitwise identical to native PyTDI, with maximum absolute difference zero. Reopened files passed exact residual and FFT identities; temporary workspaces had been removed.

12.2 Residual-only screen validation before range selection

The earlier residual-only controller snapshot tested all 5,000 blocks on the same 30-day fixture. Native noise alone fell below the measured spectral threshold everywhere, requiring no sampler or TDI and taking 2.287 seconds. One unlisted source plus native noise required two short sampler calls, recovered one source, and took 92.790 seconds including a native TDI comparison. The recorded earlier controller hash is 19733a8fa2a424101642e4cc3182f63851e4e6e6f55efbc12dee4944ca0b4655. It precedes the indexed-range addition and is not the current source identity. The evidence is tmp/subtraction_blind_20261012/scientific.json.

12.3 Forced full-pipeline noise-block benchmark

A separate benchmark bypassed the spectral and no-seed sampler gates for one native-noise block near 0.003 Hz. It searched every half-bin carrier, 32 sky positions and nine signed chirps, then ran 500 burn-in plus 1,500 production cycles with capacity 49 and four temperatures on an A100.

Stage	Time	Measured work
F-statistic search	31.690 s	14,976 trial templates; maximum $F = 11.083743$, no surviving seeds.
Unconditional RJMCMC	70.158 s	8,002 source-waveform evaluations and 8,004 likelihood calls.
Complete benchmark	102.918 s	Includes setup, covariance, geometry and result storage; excludes Python imports and fixture generation.

All 1,500 cold production draws contained zero sources, while a hotter replica briefly visited one source. The empirical modal model therefore had nothing to subtract. This one-block result is

not a calibrated false-positive rate, an independent-chain convergence check or an annual runtime measurement. The evidence is `tmp/subtraction_empty_full_20261012/result.json`.

12.4 Scope of the native comparison

The current-controller TDI check applies to the measurement arrays actually generated in the bounded recovery smoke. It verifies agreement with native PyTDI for that case. It does not establish exhaustive parity over every possible waveform, orbit, interpolation boundary, signal population or GPU. Earlier full-catalogue injection timings describe the observation producer; they are not subtraction or inference throughput.

13 Review findings and scientific limits

13.1 Search completeness and threshold calibration

The finite template bank can omit an unlisted source or miss a weak, broad or crowded source. The existing high-SNR single-source smoke does not calibrate recovery near fitted SNR 7. Template maximization, same-data covariance estimate and repeated global rounds jointly determine false alarms and misses. Validate controlled noise ensembles and injected populations across frequency, chirp, sky, crowding and range boundaries.

13.2 Endpoint and covariance health

Full-band execution reaches Nyquist, whereas the generic positive-frequency normalization and spectral reference use interior-bin assumptions. A dedicated real-endpoint comparison is needed before assigning the same statistical interpretation there. Save or independently inspect covariance eigenvalues, conditioning and estimator stability when evaluating difficult bands.

13.3 Shared rounds remain conditional fits

Each block has its own conditional posterior against a common snapshot. Previously accepted sources are frozen. Later acceptance removes already pending components, so proposal order can affect a crowded shared feature. The resulting global subtraction is coordinated across the selected blocks, but it is not one simultaneous posterior over the entire accepted catalogue. Overlapping support regions and multiple close sources require dedicated recovery and uncertainty tests.

13.4 FD proposal assessment and native TD subtraction

Fitting and acceptance use an FD response model. Accepted physical rows are then synthesized, instrument-filtered and passed through moving-arm TDI in the time domain. Numerical agreement between these routes must be measured over the intended duration, frequency and chirp range. The current smoke checks the final residual identity and a bounded recovery error; it does not bound every FD-to-TD modelling discrepancy or guarantee a global likelihood increase in a full-year crowded fit.

13.5 Reproducibility, labels and failure recovery

The output stores covariance, configuration, sample arrays and checkpoints. The sampler's separate NumPy amplitude generator and the complete software environment are not fully reconstructible from the controller configuration. Post-fit catalogue association is greedy and can alter labels, but it does not change the data-based acceptance predicate.

Opening the final path with `'w'` and incremental round writes leaves a partial result if execution fails; old output is already replaced. Temporary files remain available for diagnosis on an exception, but automatic resume or transactional publication is not implemented. Preserve successful results deliberately when rerunning a new experiment.

13.6 Stopping and residual-background interpretation

`no_new_candidates` and `no_accepted_blocks` refer to the selected range under the configured search and acceptance rules. They do not prove all resolvable DWDs have been removed, nor infer posterior convergence. A residual used for foreground or stochastic-background analysis also contains missed signals, waveform mismatch, noise-estimation effects and subtraction uncertainty. A single conditional-mean subtraction does not propagate the posterior uncertainty of the recovered source population.

14 Performance and practical use

The full FFT and covariance estimate run once per global round. Every selected block evaluates its recorded templates. RJMCMC and posterior storage follow when templates pass the F-statistic threshold. Response geometry is shared. Accepted sources from all selected blocks are synthesized and passed through native instrument filtering and GPU TDI once per committed round.

GPU synthesis and whitened response/integral evaluation accelerate numerical work. Catalogue scans, covariance smoothing, proposal/state bookkeeping, conditional amplitude draws and HDF5 I/O still involve the CPU. The pipeline is GPU assisted rather than entirely GPU resident.

At 360 days and 0.1 Hz, one float64 XYZ array is about 71.2 MiB, one complex128 real-FFT XYZ array about 71.2 MiB, and one six-link padded array about 144.0 MiB. Concurrent arrays, geometry, waveform batches, covariance and sampler state add to these base sizes. After the first attempted fit, sorting 29 million catalogue frequencies and row indices also requires hundreds of MiB of host memory, even though that scan supplies labels only.

These storage scales are deterministic array estimates. The measured 30-day benchmark and short integration smokes cannot be multiplied into a validated annual production-runtime claim.

15 Scope, source identity and coverage

The primary local file is `/Users/liangqingyuan/Desktop/codex/DWDinjection/eric_subtraction_all.py`. Its server counterpart is `/home/genshin/dwd_injection/eric_subtraction_all.py`. The walkthrough reproduces every line of the 365-line sequential Python script in its actual source order. Imports and parameter assignments are part of the Python file itself. Code text, indentation and line numbers match the source exactly; no separate presentation numbering or generated parameter block is used. Steps 4–10 repeat for each global round. Appendix A reproduces the full file, and Appendix B maps its step ranges.

The review is tied to the following source identities. Source hashes identify the inspected implementation; the bounded evidence in Section 12 defines the tested numerical scope.

eric_subtraction_all.py (365 lines)
6e34519a839f511d13ac10e370375f5d891bd4bc3710f424a20eaafd143c9db6

eric_simulation_DWD.py (134 lines)
f902efc3b63c17772d59472e1c912c134ff9f38456304d8b1157a920ab6159f9

eric_snr.py (360 lines)
7441c8ba7b7e0dc4680e5e4a14b6bb435be07684c68816f300911d46f1b0510e

eric_generate_waveform.py (266 lines)
ab9c96bbfa3de392cac9d34034aac56ed5cda45770f544f655d13d5df0a4eceb

eric_rjmcmc.py (316 lines)
9401a9332332010313416a384b44c89ca49bea7f95860e37247d8f9a3e934a46

script/dwd_blind_search.py (68 lines)
5c4cdbb56c662195ba40b20e3802c40a1e6ca9a9c66075de8618ac1f7fe3c7b0

A Complete primary-source listing

This is the complete current `eric_subtraction_all.py`, with its original line numbers.

```
'''Step 1: import all the necessary packages.'''
from pathlib import Path
from tempfile import mkdtemp
import hashlib
import json
import shutil
import h5py
import numpy as np
import lisagwresponse
import lisainstrument
import pytdi
from eric_generate_waveform import DWD_simulation_gpu, tdi_gpu
from script.eric_Fstats import (
    build_response, physical_amplitude_coordinates,
    physical_amplitudes, reconstruct_waveform, noise_inner_product,
)
from script.dwd_blind_search import (
    residual_covariance, template_f_statistic_search, amplitude_f_statistic,
)
from script.dwd_bank_association import assign_bank_ids
from eric_rjmcmc import run_eryn_rjmcmc
from script.oneblock_observation import read_checkpoint, write_checkpoint
from script.tool import _save_tree, single_cpu

'''Step 2: define the parameters.'''
orbit_file = Path('./data/dwd_observation_full_catalogue.orbits.h5')
snr_file = Path('./data/ldc_dwd_data.h5')
observation_file = Path('./data/dwd_observation_full_catalogue.h5')
output_file = Path('./data/dwd_subtraction_all.h5')
frequency_range = (0.0001, 0.05)
f_derivative_pn = [-25, -10]
freq_block_width = 1e-05
subtraction_frequency_range = None
frequency_derivative_bounds = 10. ** f_derivative_pn[1] * np.array([-1., 1.])
F_threshold = 16.0
SNR_r_threshold = 7.0
sample_number = (500, 1500)
maximum_sources = 64
with h5py.File(observation_file, 'r') as saved:
    T_obs = saved['observation'].attrs['duration']
    fs = saved['observation'].attrs['rate']
    total_xyz_td = saved['observation/total_xyz_td'][:]
    detector_config = json.loads(saved['observation'].attrs['detector_config'])
detector_config['orbits'] = str(orbit_file)
N, N_day = int(round(T_obs * fs)), int(round(86400 * fs))
frequency_blocks = np.arange(
    frequency_range[0], frequency_range[1] - freq_block_width * 1.e-10, freq_block_width,
)[:, None] + np.array([0., freq_block_width])
```

```
frequency_blocks[-1, 1] = frequency_range[1]
block_count = len(frequency_blocks)
block_indices = np.arange(block_count, dtype=np.int64)
subtraction_range = np.array([frequency_range[0], frequency_range[1]]) if subtraction_frequency_range is None
      else np.asarray(subtraction_frequency_range, dtype=float)
selected_block_indices = block_indices[
    (frequency_blocks[:, 0] >= subtraction_range[0] - freq_block_width * 1.e-10)
    & (frequency_blocks[:, 1] <= subtraction_range[1] + freq_block_width * 1.e-10)]
selected_block_count = len(selected_block_indices)
fields = dict(A=1, f=2, df=3, phi0=4, iota=5, longitude=6, latitude=7, psi_bme=8)
# Step 3. read the candidate bank and prepare shared state.
with h5py.File(snr_file, 'r') as saved:
    template_dwd = saved['catalogue'][saved['catalogue'][:, 10] > SNR_r_threshold]
total_xyz_fd, fft_frequency = np.fft.rfft(total_xyz_td, axis=-1), np.fft.rfftfreq(N, d=1 / fs)
work = Path(mkdtemp(prefix='.dwd_all_', dir=output_file.parent)).resolve()
configuration = dict(duration=T_obs, rate=fs, frequency_range=np.array([frequency_range[0], frequency_range
    [1]]),
    frequency_blocks_hz=frequency_blocks, block_indices=block_indices, frequency_block_width=freq_block_width
    ,
    subtraction_frequency_range=subtraction_range, selected_block_indices=selected_block_indices,
    derivative_bounds=frequency_derivative_bounds,
    chirp_coordinate='linear', f_threshold=F_threshold,
    relative_snr_threshold=SNR_r_threshold, sample_number=np.asarray(sample_number),
    minimum_sampler_capacity=maximum_sources)
with h5py.File(output_file, 'w') as saved:
    _save_tree(saved.create_group('config'), configuration)
    saved.create_group('blocks')
    saved.create_group('rounds')
# Step 4. start a global round and estimate covariance.
round_number, last_accepted_round = 0, 0
print(f'Subtraction range {subtraction_range[0]:.8g}--{subtraction_range[1]:.8g} Hz: '
      f'selected {selected_block_count} of {block_count} blocks, indices '
      f'{selected_block_indices[0]}--{selected_block_indices[-1]} (zero-based).', flush=True)
with single_cpu():
    while True:
        round_number += 1
        label = f'{round_number:02d}'
        print(f'Global round {round_number}: visit {selected_block_count} selected frequency blocks.', flush=
    True)
        # This complete Fourier snapshot is unchanged throughout the block loop.
        round_residual_fd = total_xyz_fd if round_number == 1 else residual_xyz_fd
        round_residual_fd.setflags(write=False)
        round_digest = hashlib.sha256(round_residual_fd.tobytes()).hexdigest()
        covariance_full = residual_covariance(round_residual_fd, T_obs, fs)
        block_order = selected_block_indices
        status = np.full(block_count, 'outside_subtraction_range', dtype='S32')
        status[selected_block_indices] = 'pending'
        with h5py.File(output_file, 'a') as saved:
            group = saved['rounds'].create_group(label)
            group.create_group('blocks')
            group.attrs.update(complete=False, committed=False, residual_input_sha256=round_digest)
        # Step 5. evaluate each selected block against its recorded templates.
        selected_counts = np.zeros(block_count, dtype=np.int64)
        accepted_counts = np.zeros(block_count, dtype=np.int64)
        proposals, proposal_doppler, proposal_coefficients, proposal_ids, proposal_blocks = [], [], [], [],
    []
        for block_index in block_order:
            key = f'{block_index:05d}'
            block_range = frequency_blocks[block_index]
            center = float(block_range.mean())
            block_derivative_bounds = np.asarray(frequency_derivative_bounds, dtype=float)
            padding = block_range[1] * 1.e-4 + max(abs(block_derivative_bounds)) * T_obs + 8 / T_obs
            retained_bins = np.flatnonzero((fft_frequency >= block_range[0] - padding)
                                           & (fft_frequency <= block_range[1] + padding) & (fft_frequency > 0)
    )
            center_bin = int(round(center * T_obs))
            wide_bins = np.arange(max(1, min(center_bin - 2048, retained_bins[0] - 256)),
                min(len(fft_frequency), max(center_bin + 2049, retained_bins[-1] + 257)))
            core_indices = retained_bins - wide_bins[0]
            physical_frequency, wide_frequency = fft_frequency[retained_bins], fft_frequency[wide_bins]
            if round_number == 1 and block_index == block_order[0]:
                builder, response_times, response_geometry = build_response(work, wide_frequency, T_obs, fs,
    orbit_file=orbit_file)
                builder_type = type(builder)
            else:
                builder = builder_type(np.broadcast_to(np.eye(3), (len(wide_frequency), 3, 3)),
```

```
                    np.zeros((3, len(wide_frequency)), complex), response_times, response_geometry,
    wide_frequency, T_obs, fs)
            template_data = dict(derivative_bounds=block_derivative_bounds,
                retained_bins=retained_bins, wide_bins=wide_bins, core_indices=core_indices,
                block_index=block_index, frequency_range=block_range,
                physical_frequency_hz=physical_frequency)
            with h5py.File(output_file, 'a') as saved:
                if key not in saved['blocks']:
                    _save_tree(saved['blocks'].create_group(key), template_data)
            covariance = covariance_full[retained_bins].copy()
            inverse = np.linalg.inv(covariance)
            covariance.setflags(write=False)
            data = round_residual_fd[:, retained_bins]
            target = (template_dwd[:, fields['f']] >= block_range[0]) & (
                (template_dwd[:, fields['f']] < block_range[1]) |
                ((block_index == block_count - 1) & (template_dwd[:, fields['f']] == block_range[1])))
            search = template_f_statistic_search(data, covariance, builder, core_indices, T_obs, fs,
                template_dwd[target], f_threshold=F_threshold)
            search['template_dwd_id'] = template_dwd[target, 0].astype(np.int64)
            selected = search['template_dwd']
            selected_counts[block_index] = len(selected)
            block_data = dict(fixed_covariance=covariance,
                screening=dict(search=search, residual_input_xyz_fd=data))
            block_attributes = dict(residual_input_sha256=round_digest,
                block_index=block_index,
                fixed_covariance_sha256=hashlib.sha256(covariance.tobytes()).hexdigest(),
                proposed_for_subtraction=False, accepted_for_subtraction=False)
            status[block_index] = 'no_new_candidates'
            # Step 6. run RJMCMC and assess fitted sources.
            if len(selected):
                print(f'  Block {key}, {block_range[0]:.8f}--{block_range[1]:.8f} Hz: fit {len(selected)}
    candidates.', flush=True)
                checkpoints = work / f'round_{label}_block_{key}'
                posterior = run_eryn_rjmcmc(data, covariance, selected, block_range,
                    block_derivative_bounds, center, T_obs, fs, checkpoints,
                    chain_count=1, inverse_temperatures=(1., .8, .6, .4),
                    iterations=sum(sample_number), burn_in=sample_number[0],
                    maximum_sources=max(maximum_sources, len(selected)), chirp_coordinate='linear',
                    response_times=response_times, response_geometry=response_geometry, physical_frequency=
    physical_frequency)
                doppler, coefficients = posterior['doppler'], posterior['fit_result']['coefficients']
                new_bases = builder.build(doppler)[:, :, :, core_indices]
                new_sources = reconstruct_waveform(coefficients, new_bases)
                source_snr = np.sqrt(np.maximum(noise_inner_product(new_sources, new_sources, inverse, T_obs,
     fs), 0.))
                assessment_data = data.copy()
                # Fits share the round snapshot, but acceptance is conditional on
                # already proposed components to avoid subtracting a sideband twice.
                if proposal_doppler:
                    points, amplitudes = np.concatenate(proposal_doppler), np.concatenate(
    proposal_coefficients)
                    nearby = abs(points[:, 0] - center) <= .5 * np.ptp(block_range) + 2 * padding
                    if np.any(nearby):
                        pending = reconstruct_waveform(amplitudes[nearby],
                            builder.build(points[nearby])[:, :, :, core_indices])
                        assessment_data -= pending.sum(axis=0)
                conditional_f = np.zeros(len(doppler))
                for i in range(len(doppler)):
                    conditional_data = assessment_data - new_sources.sum(axis=0) + new_sources[i]
                    conditional_f[i] = amplitude_f_statistic(conditional_data, new_bases[i:i + 1],
                        covariance, T_obs, fs)['two_f'][0] / 2
                owned = (doppler[:, 0] >= block_range[0]) & ((doppler[:, 0] < block_range[1])
                    | ((block_range[1] == frequency_range[1]) & (doppler[:, 0] == frequency_range[1])))
                duplicate = np.zeros(len(doppler), dtype=bool)
                previous_doppler = proposal_doppler if round_number == 1 else accepted_doppler +
    proposal_doppler
                previous_coefficients = proposal_coefficients if round_number == 1 else accepted_coefficients
     + proposal_coefficients
                if previous_doppler:
                    points, amplitudes = np.concatenate(previous_doppler), np.concatenate(
    previous_coefficients)
                    nearby = abs(points[:, 0] - center) <= .5 * np.ptp(block_range) + 2 * padding
                    if np.any(nearby):
                        previous = reconstruct_waveform(amplitudes[nearby],
                            builder.build(points[nearby])[:, :, :, core_indices])
```

```
                        overlap = noise_inner_product(new_sources[:, None], previous[None, :], inverse, T_obs, fs)
                        norm = np.sqrt(np.maximum(noise_inner_product(new_sources, new_sources, inverse, T_obs, fs)[:, None]
                            * noise_inner_product(previous, previous, inverse, T_obs, fs)[None, :], 1.e-300))
                        duplicate |= np.any(overlap / norm > .9, axis=1)
                retained = owned & ~duplicate & np.isfinite(source_snr) & (source_snr > SNR_r_threshold) & (conditional_f >= F_threshold)
                # Deduplicate the fitted components using measured waveform overlap.
                for i in np.argsort(source_snr)[::-1]:
                    if not retained[i]:
                        continue
                    for j in np.flatnonzero(retained):
                        if j == i or source_snr[j] > source_snr[i] or (source_snr[j] == source_snr[i] and j < i):
                            continue
                        overlap = noise_inner_product(new_sources[i], new_sources[j], inverse, T_obs, fs)
                        if overlap / max(source_snr[i] * source_snr[j], 1.e-300) > .9:
                            retained[j], duplicate[j] = False, True
                retained_sum = new_sources[retained].sum(axis=0)
                improvement = float(noise_inner_product(assessment_data, assessment_data, inverse, T_obs, fs)
                    - noise_inner_product(assessment_data - retained_sum, assessment_data - retained_sum, inverse, T_obs, fs))
                accepted = np.any(retained) and improvement > 0
                status[block_index] = 'proposed' if accepted else 'fit_rejected_by_data'
                # Step 7. associate recovered sources with catalogue labels.
                # Catalogue reads happen after inference and have no acceptance role.
                if round_number == 1 and np.count_nonzero(selected_counts) == 1:
                    with h5py.File(snr_file, 'r') as saved:
                        truth_frequency = saved['catalogue'][:, fields['f']]
                        chirp = saved['catalogue'][:, fields['df']]
                    maximum_drift = float(np.max(abs(chirp) * T_obs + abs(truth_frequency) * 1.e-4 + 8 / T_obs))
                    truth_order = np.argsort(truth_frequency, kind='stable')
                    truth_frequency = truth_frequency[truth_order]
                # Truth is consulted only after fitting, and cached on disk for this block.
                truth_cache = work / f'truth_{key}.h5'
                if truth_cache.exists():
                    with h5py.File(truth_cache, 'r') as saved:
                        truth_ids = saved['dwd_id'][:]
                        truth_waveforms = saved['source_xyz_fd'][:]
                else:
                    truth_low, truth_high = block_range + np.array([-2304., 2304.]) / T_obs
                    first, last = np.searchsorted(truth_frequency, [truth_low - maximum_drift, truth_high + maximum_drift])
                    nearby_indices = np.sort(truth_order[first:last])
                    with h5py.File(snr_file, 'r') as saved:
                        records = saved['catalogue'][nearby_indices]
                    frequency = records[:, fields['f']]
                    end = frequency + records[:, fields['df']] * T_obs
                    margin = np.maximum(frequency, end) * 1.e-4 + 8 / T_obs
                    nearby = (np.minimum(frequency, end) - margin <= truth_high) & (np.maximum(frequency, end) + margin >= truth_low)
                    truth_rows = records[nearby]
                    truth_ids = truth_rows[:, 0].astype(np.int64)
                    truth_doppler = truth_rows[:, [fields[name] for name in ('f', 'df', 'longitude', 'latitude')]]
                    cos_iota = np.cos(truth_rows[:, fields['iota']])
                    truth_coefficients = physical_amplitude_coordinates(truth_rows[:, fields['A']] * (1 + cos_iota**2),
                        2 * truth_rows[:, fields['A']] * cos_iota, truth_rows[:, fields['psi_bme']], truth_rows[:, fields['phi0']]) / 1.e-22
                    truth_waveforms = np.empty((len(truth_ids), 3, len(retained_bins)), dtype=complex)
                    for first in range(0, len(truth_ids), 32):
                        truth_waveforms[first:first + 32] = reconstruct_waveform(truth_coefficients[first:first + 32],
                            builder.build(truth_doppler[first:first + 32])[:, :, :, core_indices])
                    with h5py.File(truth_cache, 'w') as saved:
                        saved.create_dataset('dwd_id', data=truth_ids)
                        saved.create_dataset('source_xyz_fd', data=truth_waveforms)
                truth_association = assign_bank_ids(new_sources, truth_waveforms, truth_ids, inverse, T_obs, fs, minimum_overlap=.9)
                matched = truth_association['matched_dwd_id'].copy()
                physical = physical_amplitudes(coefficients)
                parameters = np.column_stack((physical['A'], doppler[:, 0], doppler[:, 1], physical['phi0'],
                    physical['iota'], doppler[:, 2], doppler[:, 3], physical['psi_bme']))
```

```
248                 block_data.update(catalogue_association=dict(matched_dwd_id=matched,
249                     best_overlap=truth_association['best_overlap'], scope='Post-fit labels only; unmatched
    fits may be subtracted.'),
250                     # Step 8. record the accepted fits.
251                     acceptance=dict(source_snr=source_snr, conditional_f=conditional_f,
252                         owned_by_block=owned, duplicate_previous_or_this_round=duplicate,
253                         retained=retained, likelihood_improvement=improvement),
254                     recovery=dict(doppler=doppler, coefficients=coefficients, parameters=parameters,
    source_xyz_fd=new_sources),
255                     rjmcmc={name: value for name, value in posterior.items() if name not in ('bases', '
    fit_result', 'checkpoint_file')})
256                 block_attributes.update(proposed_for_subtraction=accepted,
257                     source_xyz_fd_scope='Frequency-domain fitting and post-fit association only.')
258                 if accepted:
259                     retained_ids = matched[retained].copy()
260                     if round_number == 1 and not np.any(accepted_counts):
261                         next_unknown_id = -1
262                     if round_number == 1:
263                         used_labels = np.concatenate(proposal_ids) if proposal_ids else np.empty(0, dtype=np.
    int64)
264                     else:
265                         used_labels = np.concatenate(accepted_ids + proposal_ids)
266                     for i in range(len(retained_ids)):
267                         if retained_ids[i] < 0 or retained_ids[i] in used_labels or retained_ids[i] in
    retained_ids[:i]:
268                             retained_ids[i] = next_unknown_id
269                             next_unknown_id -= 1
270                     proposals.append(parameters[retained])
271                     proposal_doppler.append(doppler[retained])
272                     proposal_coefficients.append(coefficients[retained])
273                     proposal_ids.append(retained_ids)
274                     proposal_blocks.append(np.full(np.count_nonzero(retained), block_index, dtype=np.int64))
275                     accepted_counts[block_index] = np.count_nonzero(retained)
276                 snapshots = {}
277                 for path in sorted(checkpoints.glob('*.h5')):
278                     with h5py.File(path, 'r') as saved:
279                         snapshots[path.stem] = read_checkpoint(saved)
280                 with h5py.File(output_file, 'a') as saved:
281                     group = saved[f'rounds/{label}/blocks'].create_group(key)
282                     _save_tree(group, block_data)
283                     group.attrs.update(block_attributes)
284                     checkpoint_group = group.create_group('checkpoints')
285                     for name, snapshot in snapshots.items():
286                         write_checkpoint(checkpoint_group.create_group(name), snapshot)
287                     group['rjmcmc'].attrs['checkpoint_group'] = f'/rounds/{label}/blocks/{key}/checkpoints'
288             else:
289                 with h5py.File(output_file, 'a') as saved:
290                     group = saved[f'rounds/{label}/blocks'].create_group(key)
291                     _save_tree(group, block_data)
292                     group.attrs.update(block_attributes)
293         # Step 9. subtract the combined time-domain TDI signal.
294         if round_number == 1:
295             accepted_parameters, accepted_doppler, accepted_coefficients, accepted_ids, accepted_blocks = [],
     [], [], [], []
296             accumulated_xyz_td = np.zeros_like(total_xyz_td)
297             residual_xyz_td, residual_xyz_fd = total_xyz_td.copy(), total_xyz_fd
298         # Only here may the residual change: one waveform/TDI sum for all accepted blocks.
299         if proposals:
300             parameters = np.concatenate(proposals)
301             print(f'  Subtract {len(parameters)} fitted DWDs from {np.count_nonzero(accepted_counts)} blocks
    together.', flush=True)
302             links = DWD_simulation_gpu(parameters, detector_config)
303             response_model = lisagwresponse.GalacticBinary(A=1., f=.003, ra=0., dec=0.,
304                 orbits=str(orbit_file), orbit_interp_order=5, shift_sun='never')
305             links_file = work / 'recovered_links.h5'
306             with h5py.File(links_file, 'w') as saved:
307                 response_model.init_response_file(saved, detector_config['dt'], len(links), -2 * N_day / fs,
    'tcb')
308                 saved['tcb/y'][:] = links
309             del links
310             instrument = lisainstrument.Instrument(
311                 **{name: value for name, value in detector_config.items()
312                    if name not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', 'include_padding')
    },
313                 gws=str(links_file), size=N + 2 * N_day, t0=-N_day / fs)
314             instrument.disable_all_noises()
```

```
            instrument.disable_dopplers()
            data = pytdi.Data.from_instrument(instrument.export_numpy_full())
            new_xyz_td = tdi_gpu(data.measurements, data.args, keep=slice(N_day, N_day + N)) / instrument.
    central_freq
            accumulated_xyz_td += new_xyz_td
            residual_xyz_td = total_xyz_td - accumulated_xyz_td
            residual_xyz_fd = np.fft.rfft(residual_xyz_td, axis=-1)
            del data, instrument, response_model, new_xyz_td
            accepted_parameters.append(parameters)
            accepted_doppler.extend(proposal_doppler)
            accepted_coefficients.extend(proposal_coefficients)
            accepted_ids.extend(proposal_ids)
            accepted_blocks.extend(proposal_blocks)
            last_accepted_round = round_number
            status[status == b'proposed'] = 'accepted'
        # Step 10. record the round and decide whether to continue.
        recovered_ids = np.concatenate(accepted_ids) if accepted_ids else np.empty(0, dtype=np.int64)
        round_summary = dict(block_status=status, selected_count=selected_counts, accepted_count=
    accepted_counts,
            recovered_dwd_id=recovered_ids)
        with h5py.File(output_file, 'a') as saved:
            group = saved[f'rounds/{label}']
            _save_tree(group, round_summary)
            group.attrs.update(complete=True, committed=bool(proposals), visited_block_count=
    selected_block_count,
                selected_block_count=selected_block_count,
                accepted_block_count=int(np.count_nonzero(accepted_counts)),
                residual_output_sha256=hashlib.sha256(residual_xyz_fd.tobytes()).hexdigest())
            for block_index in np.flatnonzero(accepted_counts):
                saved[f'rounds/{label}/blocks/{block_index:05d}'].attrs.update(accepted_for_subtraction=True,
                    subtraction_backend='DWD_simulation_gpu + native Instrument + tdi_gpu')
            saved.attrs.update(last_completed_round=round_number, last_accepted_round=last_accepted_round)
        if not proposals:
            stopping_reason = 'no_new_candidates' if not np.any(selected_counts) else 'no_accepted_blocks'
            break
# Step 11. save the final results.
parameters = np.concatenate(accepted_parameters) if accepted_parameters else np.empty((0, 8))
ids = np.concatenate(accepted_ids) if accepted_ids else np.empty(0, dtype=np.int64)
final_data = dict(signal_xyz_td=accumulated_xyz_td, residual_xyz_td=residual_xyz_td,
    signal_xyz_fd=np.fft.rfft(accumulated_xyz_td, axis=-1), residual_xyz_fd=residual_xyz_fd,
    parameters=parameters, dwd_id=ids,
    doppler=np.concatenate(accepted_doppler) if accepted_doppler else np.empty((0, 4)),
    coefficients=np.concatenate(accepted_coefficients) if accepted_coefficients else np.empty((0, 4)),
    block_index=np.concatenate(accepted_blocks) if accepted_blocks else np.empty(0, dtype=np.int64))
with h5py.File(output_file, 'a') as saved:
    _save_tree(saved.create_group('final_subtraction'), final_data)
    saved['final_subtraction'].attrs['accepted_round'] = last_accepted_round
    saved.attrs.update(complete=True, single_file_complete=True, stopping_reason=stopping_reason,
        stopping_condition_met=True, iteration_stable=False)
shutil.rmtree(work)
print(f'Saved {output_file}: {round_number} global rounds, {len(parameters)} accepted components; {
    stopping_reason}.', flush=True)
pipeline_result = dict(output_file=str(output_file), rounds=round_number, source_count=len(parameters),
            selected_block_indices=selected_block_indices.tolist(), total_block_count=block_count,
            last_accepted_round=last_accepted_round, stopping_reason=stopping_reason)
```

B Complete primary-file coverage map

Step	Walkthrough lines	Source lines	Explanation
1	1–24	1–24	Import the required packages.
2	25–57	25–57	Define parameters, read the observation settings and select blocks.
3	58–73	58–73	Read the recorded template bank and prepare shared state.
4	74–95	74–95	Start a global round and estimate covariance.
5	96–144	96–144	Evaluate each selected block against its recorded templates.
6	145–204	145–204	Run RJMCMC and assess fitted sources.
7	205–249	205–249	Associate catalogue labels after fitting.
8	250–292	250–292	Record accepted fits and source IDs.
9	293–328	293–328	Subtract the combined time-domain TDI signal.
10	329–346	329–346	Record the round and decide whether to continue.
11	347–365	347–365	Save the final results and remove temporary files.

C Dependency source listings

The following files support the reviewed execution. Their line numbers belong to the named file and do not continue the primary controller's line numbers.

C.1 Observation producer: eric_simulation_DWD.py

```
# Step 1. Import packages.
import os
for thread_variable in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS', 'NUMEXPR_NUM_THREADS'):
    os.environ[thread_variable] = '1'
import importlib.metadata
import json
from pathlib import Path
import shutil
import tempfile
import time
import h5py
import lisaorbits
import lisagwresponse
import lisainstrument
import numpy as np
import pytdi
from script.oneblock_observation import pack_observation
from eric_generate_waveform import DWD_simulation_gpu, tdi_gpu


# Step 2. Configure parameter settings.
catalogue_file = Path('./data/LDC2_sangria_training_v2.h5')
observation_file = Path('./data/dwd_observation_full_catalogue.h5')
T_obs = 360. * 86400.
fs = 0.1
L = 2.5e9
N_acc = 3.0e-15
delta_x = 15.0e-12
gpu_batch_size = 10000
minimum_envelope_samples = 512
parameter_fields = dict(
    A='Amplitude', f='Frequency', df='FrequencyDerivative', phi0='InitialPhase',
```

```
    iota='Inclination', longitude='EclipticLongitude', latitude='EclipticLatitude',
    psi_bme='Polarization',
)


workspace = Path(tempfile.mkdtemp(prefix='.dwd_simulation_', dir=observation_file.parent))
orbit_file = workspace / 'orbit.h5'
N = int(round(T_obs * fs))
N_day = int(round(86400 * fs))
detector_config = dict(
    T_obs=T_obs, gpu_batch_size=gpu_batch_size,
    minimum_envelope_samples=minimum_envelope_samples, include_padding=True,
    dt=1 / fs, orbits=str(orbit_file), orbit_dataset='tcb/ltt',
    aafilter=('kaiser', 240., .275 * fs, .725 * fs),
    clock_offsets=0., clock_freqoffsets=0., clock_freqlindrifts=0., clock_freqquaddrifts=0.,
)

# Step 3. Simulate instrumental noise with the LISA packages.
lisaorbits.KeplerianOrbits(L=L).write(
    str(orbit_file), t0=-20e6, dt=86400., size=825, mode='w',
)
print('Simulating one continuous ACC+OMS noise realization.', flush=True)
noise_instrument = lisainstrument.Instrument(
    **{key: value for key, value in detector_config.items()
       if key not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', 'include_padding')}, size=N + 2
     * N_day,
    t0=-N_day / fs,
    testmass_asds=N_acc, oms_asds=(delta_x, 0., 0., 0., 0., 0.),
)
noise_instrument.disable_all_noises(excluding={'test-mass', 'oms'})
noise_instrument.disable_dopplers()
noise_data = pytdi.Data.from_instrument(
    noise_instrument.export_numpy_full(),
)

# Step 4. Simulate waveforms of all DWDs on the GPU.
with h5py.File(catalogue_file) as source:
    dgb = source['sky/dgb/cat'][:]
    igb = source['sky/igb/cat'][:]

source_count = len(dgb) + len(igb)
catalogue = np.empty(source_count, dtype=[('ID', 'i8')] + [
    (field, 'f8') for field in parameter_fields.values()
])
catalogue['ID'] = np.arange(1, source_count + 1, dtype=np.int64)
for field in parameter_fields.values():
    catalogue[field] = np.concatenate((dgb[field][:, 0], igb[field][:, 0]))
del dgb, igb

with h5py.File(workspace / 'population.h5', 'x') as saved:
    saved.create_dataset('catalogues/all', data=catalogue, chunks=True)

links_file = workspace / 'population_links.h5'
print(f'Simulating all {source_count:,} DWDs with DWD_simulation_gpu.', flush=True)
started = time.perf_counter()
links = DWD_simulation_gpu(catalogue, detector_config)
waveform_runtime = time.perf_counter() - started
print(f'GPU injection completed: {source_count:,} DWDs in {waveform_runtime:.1f}s.', flush=True)
del catalogue
response_model = lisagwresponse.GalacticBinary(
    A=1., f=.003, ra=0., dec=0., orbits=str(orbit_file),
    orbit_interp_order=5, shift_sun='never',
)
with h5py.File(links_file, 'x') as saved:
    response_model.init_response_file(saved, detector_config['dt'], len(links), -2 * N_day / fs, 'tcb')
    saved['tcb/y'][:] = links
del links

# Convert the summed links into full-duration detector measurements.
signal_instrument = lisainstrument.Instrument(
    **{key: value for key, value in detector_config.items()
       if key not in ('T_obs', 'gpu_batch_size', 'minimum_envelope_samples', 'include_padding')}, gws=str(
     links_file), size=N + 2 * N_day,
    t0=-N_day / fs,
)
signal_instrument.disable_all_noises()
```

```
signal_instrument.disable_dopplers()
signal_data = pytdi.Data.from_instrument(signal_instrument.export_numpy_full())
signal_measurements = signal_data.measurements
noise_measurements = noise_data.measurements
del noise_data, noise_instrument

# Step 5. Combine the full observation, apply TDI, and save.
total_measurements = {
    name: signal_measurements[name] + noise_measurements[name]
    for name in signal_measurements
}
tdi_results = {}
for name, measurements in (('total', total_measurements),
                           ('signal', signal_measurements), ('noise', noise_measurements)):
    tdi_results[name] = tdi_gpu(
        measurements, signal_data.args, keep=slice(N_day, N_day + N),
    ) / signal_instrument.central_freq
detector_config['orbits'] = str(observation_file.with_suffix('.orbits.h5'))
with h5py.File(workspace / 'population_tdi.h5', 'x') as saved:
    saved.attrs.update(duration=T_obs, rate=fs)
    saved.attrs['detector_config'] = json.dumps(detector_config)
    for name, value in tdi_results.items():
        saved.create_dataset(name + '_xyz_td', data=value, chunks=(3, min(N_day, N)))

pack_observation(workspace, observation_file, overwrite=True, time_domain_only=True)
shutil.copy2(workspace / 'orbit.h5', observation_file.with_suffix('.orbits.h5'))
shutil.rmtree(workspace)
print(f'Saved {source_count:,} DWDs: {observation_file.resolve()}', flush=True)
```

C.2 Measured covariance and template F-statistics: script/dwd_blind_search.py

```
"""Measured covariance and analytical F-statistics for an existing DWD template bank."""
import numpy as np


def residual_covariance(data, duration, rate, *, neighboring_bins=256, chunk_size=16384):
    """Estimate the measured XYZ covariance without rejecting any frequency blocks."""
    from script.eric_preprocess import effective_noise_cov, periodogram

    data = np.asarray(data)
    frequencies = np.arange(data.shape[-1]) / duration
    covariance = np.zeros((len(frequencies), 3, 3), complex)
    for first in range(1, len(frequencies), chunk_size):
        last = min(first + chunk_size, len(frequencies))
        lower, upper = max(1, first - neighboring_bins), min(len(frequencies), last + neighboring_bins)
        psd = periodogram(data[:, lower:upper], duration, rate)
        if not np.any(psd):
            local = np.broadcast_to(np.eye(3) * 1.e-100, psd.shape).copy()
        else:
            local = effective_noise_cov(psd, neighboring_bins=neighboring_bins, downweight_peaks=True)
            scale = np.trace(local, axis1=-2, axis2=-1).real / 3
            floor = max(float(np.median(scale)) * 1.e-10, 1.e-100)
            local += floor * np.eye(3)
        retained = local[first - lower:last - lower]
        covariance[first:last] = retained
    covariance[0] = covariance[1]
    return covariance


def amplitude_f_statistic(data, bases, covariance, duration, rate):
    """Vectorized standard real four-amplitude projection, including XYZ cross terms."""
    bases = np.asarray(bases)
    inverse = np.linalg.inv(covariance)
    factor = 4 / (duration * rate**2)
    x = factor * np.einsum('smcf,fcd,df->sm', bases.conj(), inverse, data, optimize=True).real
    M = factor * np.einsum('smcf,fcd,sndf->smn', bases.conj(), inverse, bases, optimize=True).real
    M = .5 * (M + M.swapaxes(-1, -2))
    # A rank-deficient basis still defines a projection (especially near Nyquist).
    coefficients = np.einsum('smn,sn->sm', np.linalg.pinv(M, hermitian=True, rcond=1.e-12), x)
    two_f = np.maximum(np.einsum('sm,sm->s', x, coefficients), 0.)
    return dict(two_f=two_f, coefficients=coefficients, amplitude_matrix=M)

```

```

def template_f_statistic_search(data, covariance, builder, core_indices, duration, rate,
                                template_parameters, *, f_threshold=16., batch_size=128):
    """Evaluate F at every supplied template; maximize its four amplitudes analytically."""
    templates = np.asanyarray(template_parameters)
    if templates.dtype.names:
        doppler = np.column_stack([templates[name] for name in
            ('Frequency', 'FrequencyDerivative', 'EclipticLongitude', 'EclipticLatitude')])
    elif templates.ndim == 2 and templates.shape[1] == 11:
        doppler = templates[:, [2, 3, 6, 7]]
    else:
        templates = templates.reshape(-1, 4)
        doppler = templates
    scores = np.empty(len(templates))
    coefficients = np.empty((len(templates), 4))
    for first in range(0, len(templates), batch_size):
        points = doppler[first:first + batch_size]
        padded = np.vstack((points, np.repeat(points[-1:], batch_size - len(points), axis=0)))
        bases = builder.build(padded)[:len(points), :, :, core_indices]
        fit = amplitude_f_statistic(data, bases, covariance, duration, rate)
        scores[first:first + len(points)] = fit['two_f']
        coefficients[first:first + len(points)] = fit['coefficients']
    retained = scores >= 2 * f_threshold
    return dict(template_dwd=templates[retained], doppler=doppler[retained], two_f=scores[retained],
        coefficients=coefficients[retained], template_parameters=templates,
        template_two_f=scores, retained_template_indices=np.flatnonzero(retained),
        template_evaluations=len(templates))
```

C.3 DWD synthesis and native-expression GPU TDI: eric_generate_waveform.py

```
"""GPU DWD link synthesis and native LISA time-delay interferometry."""
import numpy as np


def DWD_simulation_gpu(DWD_parameters, detector_config):
    """Return the summed six-link response of the supplied DWD parameter rows.

    DWD_parameters is a float array with shape (n_DWD, 8), with columns:
    A, f, df, phi0, iota, ecliptic longitude, ecliptic latitude, polarization.
    An (n_DWD, 11) SNR table is also accepted: ID, those eight parameters,
    SNR_absolute, SNR_relative. The ID and SNR columns are ignored.
    Alternatively, pass a one-dimensional structured table with fields:
    Amplitude, Frequency, FrequencyDerivative, InitialPhase, Inclination,
    EclipticLongitude, EclipticLatitude, Polarization.
    Additional fields such as ID and SNR are ignored. Every supplied row
    contributes once, including repeated parameter sets. No catalogue or ID
    lookup is performed; sources need not appear in an LDC file.

    detector_config requires T_obs [s], dt [s], and orbits (orbit-file path).
    The equivalent fs [Hz] and orbit_file keys remain supported. Arm lengths
    and spacecraft motion come from that orbit file. Optional settings are
    gpu_batch_size (default 10000) and minimum_envelope_samples (default 512).
    include_padding=True retains two days on each side for instrument/TDI use.

    By default, the result is a float64 NumPy array with shape (round(T_obs * fs), 6),
    sampled at t = arange(N) / fs in native LISA link order
    (12, 23, 31, 13, 32, 21). Values are dimensionless link fractional
    frequency shifts, before instrument and TDI processing. No noise is added
    and no output file is written. CUDA is required for a nonempty table.
    With include_padding=True, N_pad = round(2 * 86400 * fs) samples are retained
    at each end; the first sample is at t = -N_pad / fs.
    """
    T_obs = float(detector_config['T_obs'])
    fs = float(detector_config['fs']) if 'fs' in detector_config else 1 / float(detector_config['dt'])
    N = int(round(T_obs * fs))
    include_padding = bool(detector_config.get('include_padding', False))
    padding = int(round(2 * 86400 * fs)) if include_padding else 0
    batch_size = int(detector_config.get('gpu_batch_size', 10000))
    DWD_parameters = np.asarray(DWD_parameters)
    fields = ('Amplitude', 'Frequency', 'FrequencyDerivative', 'InitialPhase',
              'Inclination', 'EclipticLongitude', 'EclipticLatitude', 'Polarization')
    named = DWD_parameters.dtype.names is not None
    if len(DWD_parameters) == 0:
```

```
        return np.zeros((N + 2 * padding, 6), dtype=np.float64)

    from astropy import units as u
    from astropy.coordinates import BarycentricMeanEcliptic, CartesianRepresentation, SkyCoord
    from astropy.time import Time
    from script.dwd_population_gpu import PopulationLinksGPU
    # Rotate the supplied sky position and polarization basis into ICRS.
    rotation = np.empty((3, 3))
    frame = BarycentricMeanEcliptic(equinox=Time('J2000'))
    for column, axis in enumerate(np.eye(3)):
        rotation[:, column] = SkyCoord(
            CartesianRepresentation(*(axis * u.one)), frame=frame,
        ).icrs.cartesian.xyz.value
    engine = PopulationLinksGPU(
        (detector_config['orbit_file'] if 'orbit_file' in detector_config else detector_config['orbits']), T_obs, fs, batch_size=batch_size,
        minimum=int(detector_config.get('minimum_envelope_samples', 512)),
    )
    for first in range(0, len(DWD_parameters), 2 * batch_size):
        selected = DWD_parameters[first:first + 2 * batch_size]
        values = (np.column_stack([selected[field] for field in fields])
                  if named else np.asarray(selected, dtype=np.float64))
        if not named and values.shape[1] == 11:
            values = values[:, 1:9]
        longitude, latitude, polarization = values[:, 5], values[:, 6], values[:, 7]
        cos_lon, sin_lon = np.cos(longitude), np.sin(longitude)
        cos_lat, sin_lat = np.cos(latitude), np.sin(latitude)
        direction = np.column_stack((cos_lat * cos_lon, cos_lat * sin_lon, sin_lat)) @ rotation.T
        transported_u = np.column_stack((sin_lon, -cos_lon, np.zeros_like(longitude))) @ rotation.T
        ra = np.mod(np.arctan2(direction[:, 1], direction[:, 0]), 2 * np.pi)
        dec = np.arcsin(np.clip(direction[:, 2], -1., 1.))
        u_icrs = np.column_stack((np.sin(ra), -np.cos(ra), np.zeros_like(ra)))
        v_icrs = np.column_stack((-np.sin(dec) * np.cos(ra),
                                  -np.sin(dec) * np.sin(ra), np.cos(dec)))
        basis_rotation = np.arctan2(np.einsum('ij,ij->i', transported_u, v_icrs),
                                    np.einsum('ij,ij->i', transported_u, u_icrs))
        psi = np.mod(polarization + basis_rotation, np.pi)
        engine.add(np.column_stack((values[:, :5], ra, dec, psi)))
    response = engine.finish()
    return (response if include_padding else response[engine.padding:engine.padding + N]).copy()


# Lagrange interpolation adapted from PyTDI (BSD 3-Clause).
# Copyright (c) 2022, California Institute of Technology and
# Max Planck Institute for Gravitational Physics (Albert Einstein Institute).
#
# Redistribution and use in source and binary forms, with or without
# modification, are permitted provided that the following conditions are met:
# 1. Redistributions of source code must retain the above copyright notice,
#    this list of conditions and the following disclaimer.
# 2. Redistributions in binary form must reproduce the above copyright notice,
#    this list of conditions and the following disclaimer in the documentation
#    and/or other materials provided with the distribution.
# 3. Neither the name of the copyright holder nor the names of its contributors
#    may be used to endorse or promote products derived from this software
#    without specific prior written permission.
#
# THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
# AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
# IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
# ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE
# LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
# CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
# SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
# INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
# CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
# ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
# POSSIBILITY OF SUCH DAMAGE.

_CUDA_SHIFT = r'''
extern "C" __global__ void shift(
    const double *data, const double *shifts, const double *ratios,
    double *result, const long n, const int batches, const int halfp) {
    long i=(long)blockDim.x*blockIdx.x+threadIdx.x;
    if (i>=n) return;
    double q=floor(shifts[i]);
    double f=__dsub_rn(shifts[i],q);
    double taps[32];
```

```
    double factor=__dmul_rn(f,__dsub_rn(1.,f));
    for (int j=1;j<halfp;++j) {
        factor=__dmul_rn(factor,ratios[j]);
        taps[halfp-1-j]=__ddiv_rn(factor,__dadd_rn((double)j,f));
        taps[halfp+j]=__ddiv_rn(factor,__dsub_rn((double)(j+1),f));
    }
    taps[halfp-1]=__dsub_rn(1.,f);
    taps[halfp]=f;
    for (int j=2;j<halfp;++j) {
        double a=__ddiv_rn(f,(double)j);
        double b=__dsub_rn(1.,__dmul_rn(a,a));
        for (int k=0;k<2*halfp;++k) taps[k]=__dmul_rn(taps[k],b);
    }
    if (halfp>1) {
        double b=__dmul_rn(__dadd_rn(1.,f),__dsub_rn(1.,__ddiv_rn(f,(double)halfp)));
        for (int k=0;k<2*halfp;++k) taps[k]=__dmul_rn(taps[k],b);
    }
    long center=i+(long)q;
    if (center<-(halfp+1)) center=-(halfp+1);
    if (center>n+halfp-1) center=n+halfp-1;
    for (int batch=0;batch<batches;++batch) {
        double value=0.;
        for (int k=0;k<2*halfp;++k) {
            long index=center-(halfp-1)+k;
            double sample=(index>=0 && index<n)?data[(long)batch*n+index]:0.;
            value=__dadd_rn(value,__dmul_rn(taps[k],sample));
        }
        result[(long)batch*n+i]=value;
    }
}
'''


_TDI_ORBITS = {}


def tdi_gpu(mosa_links, detector_config, combination='XYZ2', *, t0=0., keep=slice(None)):
    """Return native-LISA XYZ time streams from six MOSA link measurements.

    mosa_links is an array of shape (..., N, 6), in native link order
    (12, 23, 31, 13, 32, 21). These are science-link measurements, with
    any instrument filtering already applied. The output is float64 with
    shape (..., 3, N), in X/Y/Z order and in the same units as the input.
    No FFT, instrument simulation, or file output is performed here.

    A dictionary of native PyTDI measurements is also accepted. Include
    sci_ij, tmi_ij and ref_ij when using full instrument noise: keeping these
    auxiliary beatnotes preserves the standard noise combinations. Missing
    beatnotes are zero, as in native Data.from_orbits. Keys '12', ..., '21'
    are shorthand for the six sci_ij measurements. Leading axes batch
    independent streams; the final axis of each dictionary value is time.

    detector_config requires fs [Hz] or dt [s], and orbits (or orbit_file).
    Moving unequal-arm light-travel times and derivatives come from that
    file, using orbit_dataset='tcb/ltt' by default. t0 gives the first
    input sample time. Native Data.args (fs, delays, delay_derivatives) can
    be passed instead to reuse already evaluated moving-orbit geometry.
    combination='XYZ2' selects second-generation Michelson by default;
    'XYZ1' selects first generation. keep is a slice removing input padding.
    Native orders 5/31 and FP64 arithmetic are preserved; CUDA is required.
    """
    import ctypes
    import copy
    from collections.abc import Mapping
    from pathlib import Path
    import sys
    from types import FunctionType, MethodType, SimpleNamespace
    import pytdi
    import pytdi.michelson

    channel_names = tuple(axis + combination[-1] for axis in 'XYZ')
    if isinstance(mosa_links, Mapping):
        provided = {('sci_' + str(name) if str(name) in pytdi.Data.MOSAS else str(name)):
                    np.asarray(value, dtype=np.float64) for name, value in mosa_links.items()}
    else:
        values = np.asarray(mosa_links, dtype=np.float64)
        provided = {'sci_' + mosa: values[..., column]
                    for column, mosa in enumerate(pytdi.Data.MOSAS)}
```

```
    shapes = [value.shape for value in provided.values() if value.ndim]
    shape = np.broadcast_shapes(*shapes)
    size = shape[-1]
    fs = float(detector_config['fs']) if 'fs' in detector_config else 1 / float(detector_config['dt'])
    if 'delays' in detector_config:
        native_args = detector_config
    else:
        orbit = Path(detector_config.get('orbit_file', detector_config.get('orbits', ''))).expanduser().resolve()
        stat = orbit.stat()
        dataset = detector_config.get('orbit_dataset', 'tcb/ltt')
        key = (str(orbit), stat.st_size, stat.st_mtime_ns, size, fs, float(t0), dataset)
        if key not in _TDI_ORBITS:
            data = pytdi.Data.from_orbits(str(orbit), fs=fs, t0=float(t0), dataset=dataset,
                                          sci_12=np.zeros(size, dtype=np.float64))
            _TDI_ORBITS.clear()
            _TDI_ORBITS[key] = data.args
        native_args = _TDI_ORBITS[key]

    cuda_root = Path(sys.prefix) / 'lib' / f'python{sys.version_info.major}.{sys.version_info.minor}' / 'site-packages/nvidia'
    handles = [ctypes.CDLL(str(cuda_root / package / 'lib' / library), mode=ctypes.RTLD_GLOBAL)
               for package, library in (('cuda_runtime', 'libcudart.so.12'), ('cuda_nvrtc', 'libnvrtc.so.12'))]
    import cupy as cp
    kernel = cp.RawKernel(_CUDA_SHIFT, 'shift', options=('--fmad=false',))
    ratios = {halfp: cp.asarray(np.array([(-1)*(1-j/halfp)/(1+j/halfp)
                                          for j in range(halfp)], dtype=np.float64))
              for halfp in (3, 16)}

    def timeshift(values, shifts, order=31):
        if np.isscalar(values):
            return values
        if np.isscalar(shifts) and shifts == 0:
            return values
        values = cp.ascontiguousarray(values, dtype=cp.float64)
        shifts = cp.asarray(shifts, dtype=cp.float64)
        if values.size == 1:
            return values.reshape(())
        if shifts.size != size:
            shifts = cp.broadcast_to(shifts, (size,)).copy()
        halfp = (order + 1) // 2
        result = cp.empty_like(values)
        batches = values.size // size
        kernel(((size + 127)//128,), (128,),
               (values, shifts, ratios[halfp], result, np.int64(size), np.int32(batches), np.int32(halfp)))
        return result

    gpu_dsp = SimpleNamespace(timeshift=timeshift, diff=pytdi.dsp.diff)
    methods = {}
    for name in ('build_shifts', 'build'):
        original = getattr(pytdi.TDICombination, name)
        method = FunctionType(original.__code__, {**original.__globals__, 'dsp': gpu_dsp},
                              original.__name__, original.__defaults__, original.__closure__)
        method.__kwdefaults__ = original.__kwdefaults__
        methods[name] = method
    args = {'delays': {name: cp.asarray(value, dtype=cp.float64) for name, value in native_args['delays'].items()},
            'delay_derivatives': {name: cp.asarray(value, dtype=cp.float64) for name, value in native_args['delay_derivatives'].items()},
            'fs': fs}
    names = tuple(dict.fromkeys(name for channel in channel_names
                                for name in getattr(pytdi.michelson, channel).components))
    measurements = {name: cp.asarray(np.ascontiguousarray(
        np.broadcast_to(provided.get(name, 0.), shape).reshape(-1, size))) for name in names}
    channels = []
    for name in channel_names:
        native_combination = copy.copy(getattr(pytdi.michelson, name))
        native_combination.build_shifts = MethodType(methods['build_shifts'], native_combination)
        operator = methods['build'](native_combination, **args)
        channels.append(cp.asnumpy(operator(measurements)[..., keep]))
    xyz = np.stack(channels, axis=-2)
    return xyz.reshape(*shape[:-1], 3, xyz.shape[-1])
```

C.4 Variable-count inference: eric_rjmcmc.py

```python
"""DWD priors, amplitude-marginal likelihood, and GPU-only Eryn sampling."""

from functools import lru_cache
from importlib.metadata import version
from pathlib import Path
import hashlib
import time

import jax
import jax.numpy as jnp
import jax.scipy.linalg as jsl
import numpy as np
from scipy.linalg import cho_factor, cho_solve, solve_triangular
from scipy.special import comb, gammaln
from tqdm.auto import tqdm
from eryn.backends import HDFBackend
from eryn.ensemble import EnsembleSampler
from eryn.moves import DistributionGenerateRJ, GaussianMove
from eryn.prior import ProbDistContainer
from eryn.state import State

from script.eric_generate_waveform import GPUResponse
from script.eric_preprocess import inner_product
from script.tool import single_cpu


# 1. Priors and sampled coordinates
def encode_doppler(para_doppler, f_center, T_obs, *, chirp_coordinate='log10'):
    para_doppler = np.asarray(para_doppler)
    if para_doppler.dtype.names:
        para_doppler = np.column_stack([para_doppler[name] for name in
            ('Frequency', 'FrequencyDerivative', 'EclipticLongitude', 'EclipticLatitude')])
    elif para_doppler.ndim == 2 and para_doppler.shape[1] == 11:
        para_doppler = para_doppler[:, [2, 3, 6, 7]]
    else:
        para_doppler = para_doppler.reshape(-1, 4)
    chirp = (para_doppler[:, 1]*T_obs**2 if chirp_coordinate == 'linear'
             else np.log10(para_doppler[:, 1]))
    return np.column_stack((np.log10(para_doppler[:, 0]/f_center)*f_center*T_obs*np.log(10.),
                            chirp, para_doppler[:, 2] % (2*np.pi),
                            np.sin(para_doppler[:, 3])))


def decode_doppler(para_doppler, f_center, T_obs, *, chirp_coordinate='log10'):
    para_doppler = np.asarray(para_doppler).reshape(-1, 4)
    chirp = (para_doppler[:, 1]/T_obs**2 if chirp_coordinate == 'linear'
             else 10.**para_doppler[:, 1])
    return np.column_stack((f_center*10.**(para_doppler[:, 0]/(f_center*T_obs*np.log(10.))),
                            chirp, para_doppler[:, 2] % (2*np.pi),
                            np.arcsin(np.clip(para_doppler[:, 3], -1., 1.))))


class CataloguePrior:
    def __init__(self, bounds, maximum=49):
        self.bounds, self.count_maximum = np.asarray(bounds), maximum
        self.V = np.prod(self.bounds[:, 1] - self.bounds[:, 0])
        self.DWD_para_order = [0, 1, 2, 3]
        self.key_order = self.DWD_para_order

    def logpdf(self, coords, inds, **kwargs):
        para_doppler, active = coords['dwd'], inds['dwd']
        N = active.sum(axis=-1)
        M = self.count_maximum
        V = self.V
        value = -N * np.log(V) - np.log(comb(M, N))
        outside = np.any((para_doppler < self.bounds[:, 0]) | (para_doppler > self.bounds[:, 1])
                         | ~np.isfinite(para_doppler), axis=-1) & active
        return np.where(outside.any(axis=-1), -np.inf, value)


class BirthDistribution:
    """90% initial-candidate boxes; 10% full-prior support."""
    def __init__(self, centers, bounds):
        self.centers, self.bounds = centers, bounds
        self.lower_limit = np.maximum(centers-[.2, 0., .2, .2], bounds[:, 0])
```

```
        self.upper_limit = np.minimum(centers+[.2, 0., .2, .2], bounds[:, 1])
        self.lower_limit[:, 1], self.upper_limit[:, 1] = bounds[1]
        self.lower_limit[:, 2], self.upper_limit[:, 2] = centers[:, 2]-.2, centers[:, 2]+.2

    def logpdf(self, para_doppler):
        para_doppler = np.asarray(para_doppler).reshape(-1, 4)
        broad = 1./np.prod(self.bounds[:, 1]-self.bounds[:, 0])
        if not len(self.centers):
            return np.full(len(para_doppler), np.log(broad))
        delta = (para_doppler[:, None, 2]-self.centers[:, 2]+np.pi) % (2*np.pi)-np.pi
        inside = np.all((para_doppler[:, None, [0, 1, 3]] >= self.lower_limit[:, [0, 1, 3]])
                        & (para_doppler[:, None, [0, 1, 3]] <= self.upper_limit[:, [0, 1, 3]]), axis=-1)
        inside &= np.abs(delta) <= .5*(self.upper_limit[:, 2]-self.lower_limit[:, 2])
        density = .1*broad+.9*(inside/np.prod(self.upper_limit-self.lower_limit, axis=1)).mean(axis=-1)
        return np.log(density)

    def rvs(self, size=1):
        shape = (size,) if isinstance(size, int) else tuple(size)
        para_doppler = np.random.uniform(self.bounds[:, 0], self.bounds[:, 1], size=shape+(4,))
        if len(self.centers):
            flat = para_doppler.reshape(-1, 4)
            selected = np.flatnonzero(np.random.random(len(flat)) >= .1)
            candidate = np.random.randint(len(self.centers), size=len(selected))
            flat[selected] = np.random.uniform(self.lower_limit[candidate], self.upper_limit[candidate])
        para_doppler[..., 2] %= 2*np.pi
        return para_doppler


# 2. Likelihood
def marginal_likelihood(data, bases, noise_covariance, log_determinant, T_obs, rate,
                        amplitude_prior_sigma=10.0):
    """Full 4N Gaussian integral using the noise covariance, not its inverse."""
    data_norm = inner_product(data, data, noise_covariance, T_obs, rate)
    if len(bases):
        h = bases.reshape(len(bases)*4, *data.shape)
        x = np.asarray([inner_product(a, data, noise_covariance, T_obs, rate) for a in h])
        M = np.asarray([[inner_product(a, b, noise_covariance, T_obs, rate)
                         for b in h] for a in h])
        M = 0.5 * (M + M.T)
        precision = M + np.eye(len(M)) / amplitude_prior_sigma**2
        chol, lower = cho_factor(precision, lower=True, check_finite=False)
        mean = cho_solve((chol, lower), x, check_finite=False)
        covariance = cho_solve((chol, lower), np.eye(len(M)), check_finite=False)
        penalty = np.log(np.diag(chol)).sum() + len(M)*np.log(amplitude_prior_sigma)
        improvement = 0.5 * x @ mean - penalty
        coefficients = mean.reshape(-1, 4)
        model = np.einsum('am,amck->ck', coefficients, bases, optimize=True)
    else:
        M = covariance = np.empty((0, 0))
        x = np.empty(0)
        coefficients = np.empty((0, 4))
        improvement = penalty = 0.0
        model = np.zeros_like(data)
    residual = data - model
    residual_norm = inner_product(residual, residual, noise_covariance, T_obs, rate)
    return dict(log_marginal=float(-0.5*data_norm-log_determinant+improvement),
                log_likelihood=float(-0.5*residual_norm-log_determinant),
                coefficients=coefficients, amplitude_covariance=covariance,
                projection=x, data_norm=float(data_norm), log_determinant=float(log_determinant),
                amplitude_matrix=M, log_occam_penalty=float(penalty),
                reconstructed=model, residual=residual, residual_norm=float(residual_norm))


class Likelihood:
    def __init__(self, engine, f_center, T_obs, sigma=10., *, maximum_sources=49, chirp_coordinate='log10'):
        self.f_center, self.T_obs, self.sigma = f_center, T_obs, sigma
        self.chirp_coordinate = chirp_coordinate
        self.engine, self.count_maximum = engine, maximum_sources
        self.white_data = engine.white_data
        self.data_norm = float(self.white_data @ self.white_data)
        self.waveform_evaluations, self.calls = 0, 0
        self.source = lru_cache(maxsize=256)(self._source)
        self.cached_details = lru_cache(maxsize=64)(self._details)
        self.integral = jax.jit(self._integral)
        self.zero = jax.device_put(np.zeros((4, self.white_data.size)), engine.device)

    def _source(self, key):
        para_doppler = np.frombuffer(key, dtype=np.float64)
```

```
        point = decode_doppler(para_doppler[None], self.f_center, self.T_obs,
                               chirp_coordinate=self.chirp_coordinate)[0]
        self.waveform_evaluations += 1
        return self.engine.white_single(jax.device_put(point, self.engine.device),
                                        *self.engine.arguments)

    def _integral(self, white):
        gram = white @ white.T
        lower = jnp.linalg.cholesky(gram+jnp.eye(len(white))/self.sigma**2)
        x = white @ self.white_data
        A = jsl.cho_solve((lower, True), x)
        logm = (-.5*self.data_norm+.5*x@A
                -jnp.log(jnp.diag(lower)).sum()-len(white)*jnp.log(self.sigma))
        return logm, A, lower

    def _details(self, key):
        para_doppler = np.frombuffer(key, dtype=np.float64).reshape(-1, 4)
        if not len(para_doppler):
            return -.5*self.data_norm, np.empty(0), np.empty((0, 0))
        white = [self.source(point.tobytes()) for point in para_doppler]
        white += [self.zero]*(self.count_maximum-len(para_doppler))
        logm, A, lower = self.integral(jnp.concatenate(white, axis=0))
        return float(logm), np.asarray(A)[:4*len(para_doppler)], np.asarray(lower)[:4*len(para_doppler), :4*
     len(para_doppler)]

    def details(self, para_doppler):
        return self.cached_details(np.ascontiguousarray(para_doppler, dtype=np.float64).tobytes())

    def __call__(self, para_doppler):
        self.calls += 1
        return self.details(para_doppler)[0]


# 3. Eryn samplers
class BirthDeathMove(DistributionGenerateRJ):
    def get_proposal(self, coords, inds, *args, **kwargs):
        para_doppler, new_inds, factors = super().get_proposal(coords, inds, *args, **kwargs)
        n, new_n = inds['dwd'].sum(axis=-1), new_inds['dwd'].sum(axis=-1)
        maximum = inds['dwd'].shape[-1]
        # Slot-selection ratio paired with CataloguePrior's uniform count prior.
        factors += (gammaln(n+1)+gammaln(maximum-n+1)
                    -gammaln(new_n+1)-gammaln(maximum-new_n+1))
        return para_doppler, new_inds, factors


class DWDMove(GaussianMove):
    def get_proposal(self, coords, random, branches_inds=None, **kwargs):
        active = branches_inds['dwd']
        slot = np.argmax(np.where(active, random.random(active.shape), -1.), axis=-1)
        selected = np.zeros_like(active)
        np.put_along_axis(selected, slot[..., None], active.any(axis=-1)[..., None], axis=-1)
        para_doppler, factors = super().get_proposal(coords, random, branches_inds={'dwd': selected})
        beta = self.temperature_control.betas[:, None, None, None]
        para_doppler['dwd'] = coords['dwd']+(para_doppler['dwd']-coords['dwd'])/np.sqrt(beta)
        para_doppler['dwd'][..., 2] %= 2*np.pi
        return para_doppler, factors


def run_eryn_rjmcmc(data, covariance, bank_doppler,
        frequency_range, derivative_bounds, f_center, T_obs, rate,
        checkpoint_directory, *, response_times, response_geometry, physical_frequency,
        chain_count=1, inverse_temperatures=(1., .8, .6, .4),
        iterations=6000, burn_in=1000, maximum_sources=49, amplitude_prior_sigma=10.,
        chirp_coordinate='log10'):
    """Fit one residual with fixed covariance on GPU; the caller owns outer rounds."""
    with single_cpu():
        started = time.perf_counter()
        maximum, sigma = maximum_sources, amplitude_prior_sigma
        betas = np.asarray(inverse_temperatures)
        checkpoint_file = Path(checkpoint_directory)/'eryn_chain_00.h5'
        covariance = np.array(covariance, dtype=np.complex128, copy=True)
        covariance_sha = hashlib.sha256(covariance.tobytes()).hexdigest()
        scale = f_center*T_obs*np.log(10.)
        chirp_bounds = np.asarray(derivative_bounds, dtype=float)
        chirp_bounds = chirp_bounds*T_obs**2 if chirp_coordinate == 'linear' else np.log10(chirp_bounds)
        bounds = np.array([np.log10(np.asarray(frequency_range)/f_center)*scale,
                           chirp_bounds, [0., 2*np.pi], [-1., 1.]])
        centers = encode_doppler(bank_doppler, f_center, T_obs, chirp_coordinate=chirp_coordinate)
```

```
231
232         engine = GPUResponse(covariance, data, response_times, response_geometry,
233                              physical_frequency, T_obs, rate)
234         likelihood = Likelihood(engine, f_center, T_obs, sigma, maximum_sources=maximum,
235                                 chirp_coordinate=chirp_coordinate)
236         checkpoint_file.parent.mkdir(parents=True, exist_ok=True)
237         prior = CataloguePrior(bounds, maximum)
238         within = DWDMove({'dwd': np.diag(np.array([.025, .3, .015, .015])**2)})
239         births = ProbDistContainer({(0, 1, 2, 3): BirthDistribution(centers, bounds)})
240         jump = BirthDeathMove({'dwd': births}, nleaves_min={'dwd': 0},
241                               nleaves_max={'dwd': maximum}, prevent_swaps=True)
242         backend = HDFBackend(str(checkpoint_file))
243         sampler = EnsembleSampler(
244             1, {'dwd': 4}, likelihood, {'dwd': prior, 'all_models_together': prior},
245             branch_names=['dwd'], nleaves_min={'dwd': 0}, nleaves_max={'dwd': maximum},
246             moves=within, rj_moves=jump, pool=None, backend=backend,
247             tempering_kwargs=dict(betas=betas, adaptive=False, permute=False),
248             fill_zero_leaves_val=likelihood(np.empty((0, 4))),
249         )
250         para_doppler = np.zeros((len(betas), 1, maximum, 4))
251         active = np.zeros(para_doppler.shape[:-1], dtype=bool)
252         para_doppler[:, 0, :len(centers)], active[:, 0, :len(centers)] = centers, True
253         state = State({'dwd': para_doppler}, inds={'dwd': active})
254         initial_counts = active[:, 0].sum(axis=-1)
255         initial_logm = np.full(len(betas), likelihood(centers))
256         initial_prior = prior.logpdf(state.branches_coords, state.branches_inds)[:, 0]
257         setup_seconds = time.perf_counter()-started
258
259         para_doppler_draws, a_draws, count_draws, logm_draws = [], [], [], []
260         amplitude_rng = np.random.default_rng()
261         progress = tqdm(sampler.sample(state, iterations=iterations), total=iterations,
262                         desc='Eryn RJMCMC', unit='cycle', dynamic_ncols=True)
263         for cycle, state in enumerate(progress, 1):
264             progress.set_postfix(N=state.branches_inds['dwd'][:, 0].sum(axis=-1).tolist(),
265                                  refresh=False)
266             if cycle <= burn_in:
267                 continue
268             para_doppler = state.branches_coords['dwd'][0, 0, state.branches_inds['dwd'][0, 0]]
269             logm, A, lower = likelihood.details(para_doppler)
270             draw = A+(solve_triangular(lower.T, amplitude_rng.normal(size=len(A)),
271                                        lower=False) if len(A) else A)
272             order = np.argsort(para_doppler[:, 0])
273             padded_para_doppler, padded_a = np.full((maximum, 4), np.nan), np.full((maximum, 4), np.nan)
274             padded_para_doppler[:len(para_doppler)], padded_a[:len(para_doppler)] = para_doppler[order], draw.
    reshape(-1, 4)[order]
275             para_doppler_draws.append(padded_para_doppler)
276             a_draws.append(padded_a)
277             count_draws.append(len(para_doppler))
278             logm_draws.append(logm)
279
280         probability = np.bincount(count_draws, minlength=maximum+1)/len(count_draws)
281         modal = int(np.argmax(probability))
282         selected = np.flatnonzero(np.asarray(count_draws) == modal)
283         best = selected[np.argmax(np.asarray(logm_draws)[selected])]
284         doppler = decode_doppler(para_doppler_draws[best][:modal], f_center, T_obs,
285                                  chirp_coordinate=chirp_coordinate)
286         bases = engine.build(doppler)
287         fit = marginal_likelihood(data, bases, covariance, 0., T_obs, rate, sigma)
288         counts = np.vstack((initial_counts, backend.get_nleaves()['dwd'][:, :, 0]))
289         logm = np.vstack((initial_logm, backend.get_log_like()[:, :, 0]))
290         logp = np.vstack((initial_prior, backend.get_log_prior()[:, :, 0]))
291         chain = dict(sampled_q=np.array(para_doppler_draws), sampled_amplitudes=np.array(a_draws),
292             source_count=np.array(count_draws), log_marginal=np.array(logm_draws),
293             trace_counts=counts[1:], trace_logm=logm[1:], initial_replica_counts=initial_counts,
294             cycle_state=dict(cycle=np.arange(iterations+1),
295                 is_burn_in=np.arange(iterations+1) <= burn_in, source_count=counts,
296                 log_marginal=logm, log_prior=logp, log_posterior=logm*betas+logp))
297         result = dict(doppler=doppler, bases=bases, fit_result=fit, modal_count=modal,
298             count_probability=probability, count_support=np.arange(maximum+1), chains={'0': chain},
299             candidate_centers=centers, initial_replica_counts=initial_counts,
300             config=dict(bounds=bounds, maximum_sources=maximum, center=f_center, duration=T_obs,
301                 rate=rate, iterations=iterations, burn_in=burn_in, thin=1, chain_count=1,
302                 inverse_temperatures=betas, amplitude_prior_sigma=sigma,
303                 chirp_coordinate=chirp_coordinate,
304                 initialization='all_candidates', initial_selection=None,
305                 use_gpu=True, backend='gpu',
306                 independent_chain_convergence_assessed=False),
307             engine='Eryn '+version('eryn'), checkpoint_file=str(checkpoint_file),
```

```
            fixed_covariance_sha256=covariance_sha, backend_setup_seconds=setup_seconds,
            runtime_seconds=time.perf_counter()-started,
            waveform_evaluations=likelihood.waveform_evaluations, likelihood_calls=likelihood.calls,
            count_prior='Uniform N=0..maximum_sources; exchangeable active slots.',
            amplitude_prior=f'Normalized zero-mean Gaussian sigma={sigma} on each A_mu/1e-22.',
            proposal='Eryn Gaussian MH and distribution RJ; single-source updates; 90% candidate-box births.'
    ,
            estimate='Best cold marginal likelihood at modal N; conditional-mean amplitudes.')
        result.update(gpu_device=str(engine.device), gpu_device_kind=engine.device.device_kind)
        return result
```

D Review reproduction

Runtime validation guards and explicit exceptions have been removed. The waveform, TDI and statistical calculations retain their valid-input operations. The TeX embeds source snapshots and is compilable without accessing the server.

```
python -m unittest discover -s tests -p test_subtraction_all.py
python -m unittest discover -s tests -p test_blind_search.py
python -m unittest discover -s tests -p test_tdi_subtraction.py
# In the server CUDA scientific environment, using the saved native fixture:
python tests/smoke_subtraction_range.py
```

The source-listing checker compares every marked TeX listing with its named Python file and verifies complete primary-file and SNR-producer coverage, current source hashes, unique section labels and resolved cross-references. The PDF is separately compiled and rendered to check title, contents, code, tables, mathematical sections and page transitions.